

THE UNIVERSITY OF ZAMBIA
SCHOOL OF NATURAL AND APPLIED SCIENCES
DEPARTMENT OF COMPUTING AND INFORMATICS

FINAL YEAR PROJECT REPORT
ACADEMIC YEAR 2026

SVDS: A Camera-Based Stolen Vehicle Detection System for the Zambia Police Service

The Backend, Database and Security Component

A Final Year Project Report Submitted to the
Department of Computing and Informatics
in partial fulfilment of the requirements for the award of

[DEGREE PROGRAMME]

Student Name: Moomba Nicholas Katapazi
Computer Number: 2021473333
Supervisor: [SUPERVISOR NAME]
Date of Submission: [DATE OF SUBMISSION]

DECLARATION

I declare that this final year project report is my own work and has been prepared in accordance with the academic requirements of the University of Zambia. All sources of information, ideas, data, software, images, and other material that are not my own have been appropriately acknowledged and referenced.

Where this project was undertaken as part of a group, I have clearly identified my individual contributions and have not presented the work of other group members as my own.

Student Name: Moomba Nicholas Katapazi

Computer Number: 2021473333

Signature: _____

Date: _____

APPROVAL

This final year project report has been submitted for examination with the approval of the project supervisor.

Supervisor Name

Signature

Date

Departmental/Examiner Approval (if applicable)

ABSTRACT

Stolen-vehicle recovery in Zambia depends largely on police officers chancing to recognise a plate number from a paper report or radio bulletin. This project, carried out by a group of four students, built SVDS, a Stolen Vehicle Detection System on top of the VRRS vehicle registry. The public files stolen-vehicle reports through a web application, and a camera node reads plates from a phone's video stream and checks them against active reports, so connected police officers are alerted in real time.

This individual report covers the backend and security component: the FastAPI service, its PostgreSQL data model, and its authentication and access control. The backend has four tables (users, reports, alerts, audit log), stores passwords as bcrypt hashes, issues signed JSON Web Tokens that carry the user's role, and enforces three roles (reportee, police, admin) on the server with reusable dependencies plus per-report ownership rules. It implements the report lifecycle (under review, missing, found), an audit trail, and the endpoint the camera node calls to ask whether a plate is stolen, which matches plates regardless of case and spacing.

The backend was verified with 31 automated tests, run against an in-memory SQLite database. Targeted probe tests then confirmed weaknesses that the tests did not cover: tokens remained valid after a user was deactivated, demoted or logged out; report status accepts arbitrary values; the public plate-check endpoint can be called repeatedly to create false alerts; and failed logins are not limited. The most serious of these, stale tokens after deactivation or demotion, was then fixed in the code (every request now re-checks the user in the database), and the probes were re-run against the fixed code: F-1 and F-2 no longer reproduce, and the suite now has 40 passing tests. Seven of the ten recorded findings remain open. Four were re-demonstrated in the post-fix probes (report status validation, the unauthenticated plate-check endpoint, login throttling and logout); three concern duplicate pending reports, the default signing-key fallback and the administrator seed script. The report concludes that the backend now meets its access-control goals, but should not be used with real police data until the remaining recommendations are applied.

Keywords: role-based access control; JSON Web Token; FastAPI; PostgreSQL; API security

ACKNOWLEDGEMENTS

[Write acknowledgements here: supervisor, group members (AI/vision, real-time and integration, frontend), family, and anyone who helped collect plate images. The author must write this; it is personal.]

CONTENTS

DECLARATION	i
APPROVAL	ii
ABSTRACT	iii
ACKNOWLEDGEMENTS	iv
LIST OF FIGURES	ix
LIST OF TABLES	x
LIST OF ABBREVIATIONS AND ACRONYMS	xi
NOTE ON GROUP PROJECTS	xii
EVALUATED CODE VERSIONS	xiii
Chapter 1: Introduction	1
1.1 Background and Context	1
1.2 Problem Statement	1
1.3 Project Aim	1
1.4 Project Objectives	2
1.5 Research Questions	2
1.6 Scope of the Project	3
1.7 Significance of the Project	3
1.8 Limitations	3
1.9 Organisation of the Report	3
Chapter 2: Literature Review	4
2.1 Introduction	4
2.2 Key Concepts and Theories	4
2.3 Existing Systems / Related Work	5
2.4 Comparative Analysis	5
2.5 Research / Knowledge Gap	6
2.6 Conceptual Framework	6
2.7 Chapter Summary	6
Chapter 3: Methodology	7
3.1 Introduction	7
3.2 Research / Development Approach	7
3.3 Requirements / Data Collection	7
3.4 System Architecture / Research Workflow	7
3.5 Tools and Technologies	8
3.6 Ethical and Legal Considerations	9

3.7 Evaluation Method	9
3.8 Chapter Summary	9
Chapter 4: System Analysis and Design	10
4.1 Introduction	10
4.2 Stakeholder and User Analysis	10
4.3 Functional Requirements	10
4.4 Non-Functional Requirements	11
4.5 Use Cases / User Stories	11
4.6 System / Solution Architecture	12
4.7 Database / Data Design	13
4.8 Interface / Interaction Design	16
4.9 Algorithms / Models / Technical Design	22
4.10 Chapter Summary	24
Chapter 5: Implementation / Development	25
5.1 Introduction	25
5.2 Development Environment	25
5.3 System Components / Modules	25
5.4 Key Implementation Details	26
5.5 Integration	28
5.6 Challenges and Solutions	28
5.7 Individual Contribution (group project)	29
5.8 Chapter Summary	29
Chapter 6: Testing, Results and Evaluation	30
6.1 Introduction	30
6.2 Testing Strategy	30
6.3 Test Cases and Results	31
6.4 Security Findings from Probes	32
6.5 User Evaluation	35
6.6 Analysis of Results	35
6.7 Objective-by-Objective Evaluation	36
6.8 Individual Results	36
6.9 Chapter Summary	36
Chapter 7: Discussion	37
7.1 Introduction	37
7.2 Interpretation of Findings	37
7.3 Comparison with Existing Work	37

7.4 Contributions of the Project	38
7.5 Limitations and Implications	38
7.6 Chapter Summary	38
Chapter 8: Conclusion and Recommendations	39
8.1 Introduction	39
8.2 Summary of the Project	39
8.3 Conclusions	39
8.4 Achievement of Objectives	39
8.5 Recommendations	40
8.6 Future Work	40
References	41
Appendix A: Individual Contribution Statement (Group Projects)	42
Purpose of this Statement	42
A.1 Project and Group Information	42
A.2 Overall Group Project	42
A.3 Individual Role and Responsibilities	42
A.4 Detailed Individual Contribution	43
A.5 Individual Deliverables	43
A.6 Contribution to the Main System or Research Output	44
A.7 Individual Implementation and Technical Work	44
A.8 Individual Testing and Evaluation Contribution	44
A.9 Collaboration with Other Group Members	44
A.10 Contribution Summary	44
A.11 Estimated Contribution	44
A.12 Statement of Authorship and Contribution	45
A.13 Supervisor Verification	45
Appendix B: User Manual / Installation Guide	46
B.1 Requirements	46
B.2 Install and configure	46
B.3 Run	46
B.4 Roles at a glance	47
B.5 Troubleshooting	47
B.6 Run the tests	47
Appendix C: Additional System Designs / Diagrams	48
C.1 Authentication sequence	48
C.2 Role-based route access	49

C.3 Database schema	50
C.4 Detection-to-alert sequence	51
Appendix D: Additional Test Cases and Results	52
D.1 Raw test output	52
D.2 Probe results (raw)	52
D.2a Probe results after the fix (raw)	52
D.3 One probe as an example	52
Appendix E: Data Collection Instruments	53
Appendix F: Additional Code / Configuration	54
F.1 Role dependencies (app/middleware.py)	54
F.2 Where finding F-1 and F-2 come from	54
F.3 Environment template (.env.example)	54
Appendix G: Other Supporting Material	55
G.1 Group role split	55
G.2 Findings summary	55

LIST OF FIGURES

Figure 3.1: SVDS system architecture	8
Figure 4.1: detection-to-alert sequence, which the backend's check-plate endpoint serves	12
Figure 4.2: entity-relationship diagram	13
Figure 4.3: live database schema	15
Figure 4.4: the generated API documentation at /docs - detail panel 1 of 4	18
Figure 4.4: the generated API documentation at /docs - detail panel 2 of 4	19
Figure 4.4: the generated API documentation at /docs - detail panel 3 of 4	20
Figure 4.4: the generated API documentation at /docs - detail panel 4 of 4	21
Figure 4.5: authentication sequence	22
Figure 4.6: per-request authorisation after the fix	23
Figure 4.7: role-based route access	24
Figure C.1: authentication sequence	48
Figure C.2: role-based route access	49
Figure C.3: entity-relationship diagram	50
Figure C.4: detection-to-alert sequence	51

LIST OF TABLES

Table P.1: List of abbreviations and acronyms	xi
Table 1.1: Individual objectives (backend and security)	2
Table 2.1: Comparative analysis	5
Table 3.1: Tools and technologies	8
Table 4.1: Stakeholder and user analysis	10
Table 4.2: Functional requirements	10
Table 4.3: Database / data design	13
Table 4.4: Tables in the live database	15
Table 4.5: Interface / interaction design	16
Table 5.1: Development environment	25
Table 5.2: System components / modules	25
Table 5.3: Challenges and solutions	28
Table 6.1: Testing strategy	30
Table 6.2: Automated test cases	31
Table 6.3: Findings	32
Table 6.4: Finding status at commit b326b21	34
Table 6.5: Objective-by-objective evaluation	36
Table 7.1: Limitations and implications	38
Table 8.1: Achievement of objectives	39
Table A.1: Project and group information	42
Table A.2: Individual role and responsibilities	42
Table A.3: Individual deliverables	43
Table A.4: Estimated contribution	45
Table B.1: Install and configure	46
Table B.2: Roles at a glance	47
Table B.3: Troubleshooting	47
Table G.1: Group role split	55
Table G.2: Findings summary	55

LIST OF ABBREVIATIONS AND ACRONYMS

Table P.1: List of abbreviations and acronyms

Abbreviation	Meaning
ALPR	Automatic Licence Plate Recognition
API	Application Programming Interface
CUDA	Compute Unified Device Architecture (NVIDIA GPU computing platform)
CORS	Cross-Origin Resource Sharing
ORM	Object-Relational Mapper
OWASP	Open Worldwide Application Security Project
RBAC	Role-Based Access Control
REST	Representational State Transfer
DBMS	Database Management System
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
IoU	Intersection over Union
JWT	JSON Web Token
mAP	mean Average Precision
MJPEG	Motion JPEG (video stream format)
ML	Machine Learning
OCR	Optical Character Recognition
SVDS	Stolen Vehicle Detection System
UI	User Interface
VRRS	Vehicle Registry and Reporting System
YOLO	You Only Look Once (object detector family)

NOTE ON GROUP PROJECTS

Students who worked on a project as a group are strongly encouraged to prepare their own individual final reports rather than submitting identical reports. The project, system, dataset, or overall research problem may be shared, but each student's report should clearly demonstrate that student's own understanding, responsibilities, work, results, and contribution to the project.

In particular, group-project students should:

- state the overall group project and explain how the individual work fits within it;
- clearly identify their own responsibilities and deliverables;
- focus detailed analysis, design, implementation, experiments, testing, and/or evaluation on their own contribution;
- acknowledge the work completed by other group members where it is discussed;
- include an *Individual Contribution Statement* in the appendices; and
- not claim another group member's work, results, code, data collection, or design decisions as their own.

EVALUATED CODE VERSIONS

Note on the version of the code evaluated. The findings F-1 to F-10 were first obtained against the code as committed at commit a349fc6 (31 tests; line counts and coverage figures in this report are from that commit). The code was then changed and committed as b326b21 (middleware.py, alerts.py, system.py, main.py, conftest.py, a new tests/test_regressions.py, and two front-end files with a new front-end test). Against that commit the backend suite has **40 tests, all passing** (thesis/assets/test-results/backend-pytest-output.txt), and the front-end suite has one test, which also passes. The original probes were then **re-run** against the fixed code (Table 6.4 in §6.4.1).

Fixed and re-verified: **F-1** and **F-2** (every protected request now re-reads the user from the database, so a deactivated, demoted or deleted user's old token is rejected with 401 or 403, and the role comes from the database, not the token) and **F-9** (CORS_ORIGINS is now read). The same check now also protects the WebSocket and the live-feed proxy. *Still open, re-confirmed by re-running the probes:* F-3, F-4, F-6 and F-7. *Still open, not re-probed:* F-5 (TC-19 still passes), F-8 (the fallback is unchanged) and F-10 (the script is unchanged).

Chapter 1: Introduction

1.1 Background and Context

Vehicle theft is a persistent problem for law enforcement in Zambia. Recovering a stolen vehicle depends heavily on chance: an officer at a checkpoint or on patrol happens to recognise a plate number from a paper report or a radio bulletin. Vehicle records, incident reports and eyewitness descriptions are typically kept on paper or in disconnected station-level spreadsheets, so a report filed at one police station is invisible to an officer elsewhere.

This report describes **SVDS**, the Stolen Vehicle Detection System, built by a group of four students on top of the **VRRS** (Vehicle Registry and Reporting System). Members of the public file stolen-vehicle reports through a web application. A camera node reads number plates from a phone's video stream and asks the server whether each plate belongs to an active report. Police officers are alerted when it does.

Every part of that flow depends on one server-side component: the **backend**. It stores the reports and accounts, decides who may see and change what, and answers the camera's question "is this plate stolen?". This report is the individual report of the student responsible for that component and for **security and access control**.

1.2 Problem Statement

Stolen-vehicle information in Zambia is fragmented and slow to reach the people who could act on it. A system that lets the public report a theft and lets cameras check plates against those reports needs a central service that:

1. keeps one authoritative record of reports, users and detections;
2. is trustworthy about *who* is asking, so that public users, officers and administrators each see only what their role permits; and
3. keeps a record of who did what, so that decisions about a report can be traced.

Without these, a reporting system either leaks sensitive vehicle and personal data, or lets the wrong person activate, alter or delete a report. The specific problem addressed here is **designing and building a database-backed API that stores reports securely, enforces role-based access on the server, and answers plate-match queries from the camera node**.

1.3 Project Aim

Group aim. To design, implement and evaluate SVDS: a vehicle registry and reporting web application integrated with a real-time, camera-based licence plate detection pipeline, for the Zambia Police Service and the public it serves.

Individual aim (Backend and security). To build and evaluate the backend API, its PostgreSQL data model, and its authentication and access-control layer.

1.4 Project Objectives

Group objectives

1. Role-based accounts (Reportee, Police, Admin) with scoped access.
2. Public filing and tracking of stolen-vehicle reports (under_review -> missing -> found).
3. A live view of alerts, the registry and analytics for police.
4. Administrator control of accounts, system health and the audit trail.
5. Detection and reading of plates from a live camera feed, matched against active reports.
6. Real-time alerts to connected police clients.

Individual objectives (Backend and security)

Table 1.1: Individual objectives (backend and security)

No.	Objective	Measure of success
B1	Design and implement the relational data model (users, reports, alerts, audit log)	Four tables with keys and constraints created from the ORM models
B2	Implement registration, login and password storage without plaintext passwords	Passwords stored as bcrypt hashes; login issues a signed, expiring JWT
B3	Enforce three-role access control on the server for every route	A test matrix shows each role reaching only its permitted routes
B4	Implement the report lifecycle with validation and ownership rules	Reportees see only their own reports; only police can activate or resolve; only admins can delete
B5	Implement the plate-match endpoint used by the camera node	Tolerates case and spacing differences; matches only active reports; records an alert
B6	Write an audit record for state-changing actions	Each such action produces a row naming the actor and target
B7	Verify the above with automated tests and a security review	All backend tests pass; weaknesses found are recorded honestly

1.5 Research Questions

1. Can role-based access control be enforced consistently on the server, using a signed token that carries the user's role, so that no route depends on the front end to hide it?
2. What security weaknesses remain in such a design once it works, and which of them can be demonstrated rather than merely suspected?

1.6 Scope of the Project

1.6.1 In Scope

- **This report:** the FastAPI application structure, the SQLAlchemy models and PostgreSQL schema, the schemas that validate input and output, registration and login, password hashing, JWT creation and checking, the role-checking dependencies, user management, the report lifecycle endpoints, the check-plate endpoint, audit logging, and the tests of these.
- **Group:** the whole system, including the components in Chapter 5's table that belong to other members.

1.6.2 Out of Scope

- The detection model and camera node (AI/vision member).
- The full WebSocket, camera-status, live-feed and health implementations (real-time and integration work). This report includes their shared token-validation boundary and relevant regression results; it does not claim their complete implementation.
- The React portals and analytics charts (frontend member).
- Production hosting, TLS termination, backups and database migrations.

1.7 Significance of the Project

The backend provides the storage and access-control foundation for the SVDS prototype. Its intended benefit is a shared record that authorised officers can search, with an audit trail for many account and report changes. Deployment by the Zambia Police Service and complete action traceability have not been demonstrated. The public gain a way to file a report and follow its status without being able to see anyone else's. The project also documents, honestly, where a small role-based API of this kind is weak, which is useful to anyone extending it.

1.8 Limitations

- Security testing was limited to automated tests and targeted probes run against an in-memory test database. No penetration test, load test or external review was carried out.
- The tests run on SQLite, not on the PostgreSQL database used in practice. One PostgreSQL-specific function is emulated in the test set-up (§6.2).
- No formal stakeholder interviews are recorded, so requirements are the group's own analysis.
- Of the ten weaknesses found (§6.4), three were fixed and re-verified (F-1, F-2, F-9); the other seven were **documented, not fixed**, within the project's time.

1.9 Organisation of the Report

Chapter 2 reviews the concepts (REST, JWT, bcrypt, RBAC) and related practice. Chapter 3 describes the method, tools, ethics and evaluation. Chapter 4 gives the analysis and design of the backend. Chapter 5 describes the implementation and the student's contribution. Chapter 6 presents the tests, the security probes and an objective-by-objective evaluation. Chapter 7 discusses the findings. Chapter 8 concludes. The appendices contain the Individual Contribution Statement, installation guide, extra diagrams, test output and code.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the concepts behind the backend: resource-oriented APIs, relational data, password hashing, signed tokens, role-based access control and audit logging. It compares selected documented capabilities with the implemented prototype. Numbered references identify the sources; implementation claims are checked against the local repository.

2.2 Key Concepts and Theories

2.2.1 REST APIs and FastAPI

REST is an architectural style defined by constraints, not simply the use of HTTP verbs [10]. SVDS uses resource-oriented routes for reports, users and alerts, but this report does not establish full REST conformance. FastAPI supports typed request handling, dependency injection and generated interactive API documentation [1]. Pydantic models validate the declared input structure [4]. Validation is only as restrictive as those declarations and the route logic: declaring `status` as a string does not enforce a lifecycle.

2.2.2 Relational databases and ORMs

PostgreSQL provides relational tables and constraints [3]. SQLAlchemy maps application classes to database structures and supports ORM queries [2]. In this project, `create_all()` creates missing tables but is not a migration mechanism for altering an existing schema. This distinction matters to the dated model/database discrepancy reported in section 4.7.1. The SQLite test database exercises much of the application logic, but cannot establish every PostgreSQL-specific behaviour.

2.2.3 Password hashing

Password storage should use a purpose-built, salted password hash. The original bcrypt work describes an adaptable computation cost intended to make offline guessing more expensive as hardware improves [6]. Passlib provides the `CryptContext` interface used by this backend [7]. Hashing does not compensate for weak passwords or unrestricted online guesses. The six-character minimum and missing login throttling are assessed separately against NIST guidance in Chapter 6 [15].

2.2.4 JSON Web Tokens

RFC 7519 defines JWT claims, including the subject (`sub`) and expiry (`exp`) [5]. JWTs can be signed or encrypted; SVDS specifically uses signed HS256 tokens. A valid signature establishes that token contents have not been altered without the signing key; it does not establish that the account is still active or that its role is unchanged.

The initial backend trusted the role claim until expiry. The corrected backend also looks up the current account on protected requests and authorises using its database role. Its authorisation decisions therefore depend on current server-side state even though the credential is a JWT. Logout remains a separate issue: returning a success message does not revoke an already-issued token.

2.2.5 Role-based access control

RBAC assigns permissions through roles rather than defining each user's permissions independently [9]. SVDS has reportee, police and admin roles, with broader privileges for police and admin. These are implemented as explicit allowed-role sets, not a database role hierarchy. Row-level ownership checks additionally restrict reportee access to reports. Public endpoints remain outside these protected-route checks.

2.2.6 Audit logging

The project's audit table records selected account and report actions with actor and target identifiers. Such records support investigation, but the implemented trail is incomplete: failed logins are not logged, alert creation is not itself an audit-log entry, and several business changes and audit inserts use separate commits. An audit table alone therefore does not establish complete or atomic traceability.

2.3 Existing Systems / Related Work

- **Django REST framework** documents pluggable authentication approaches and related permission mechanisms [12]. Its authentication framework is a useful comparison; it is not evidence that a default installation meets this project's requirements.
- **Keycloak** documents central identity administration, session management, refresh tokens and configurable multi-factor authentication [13]. Those capabilities are broader than the project's login/JWT implementation. SVDS is not an OAuth 2.0 or OpenID Connect identity-provider equivalent merely because it issues JWTs.
- **OWASP Top 10:2021 and API Security Top 10:2023** provide the named risk taxonomies used to organise the review [8], [14]. They guide questions about access, authentication, configuration and resource use. Mapping findings to them does not constitute certification or an exhaustive assessment.
- **Police-system comparison:** no independently evaluated police registry was included. The report therefore makes no comparative security or recovery-performance claim about existing police systems.

2.4 Comparative Analysis

Table 2.1: Comparative analysis

Criterion	This project	Django REST framework	Keycloak
Authentication basis	Local passwords and signed JWTs	Pluggable authentication [12]	Configurable identity service [13]
Current-account checks	Added after initial findings	Depends on chosen authentication implementation	Session and account management available
Refresh / revocation	Not implemented	Depends on scheme or integration	Documented session/token capabilities
Multi-factor authentication	Not implemented	Requires chosen integration	Configurable

Criterion	This project	Django REST framework	Keycloak
Production suitability	Not established; open findings	Depends on application and deployment	Depends on secure configuration and integration

The table compares documented capabilities with code-observed features. No comparative performance measurements or setup-effort experiment was conducted. A library or identity provider does not, by itself, make a deployment secure.

2.5 Research / Knowledge Gap

The project's practical contribution is a working backend and a documented examination of its access-control behaviour in a stolen-vehicle reporting workflow. This is a project-specific engineering contribution, not a claim that the broader literature lacks JWT security analysis. The initial and corrected versions show why current-account checks, lifecycle validation and logout invalidation must be considered separately.

2.6 Conceptual Framework

The following is a conceptual view of a protected request, not a claim about the exact order of FastAPI's internal validation and dependency execution:

```
Request + token -> validate token -> read active database user
                  -> apply current role and ownership rules
Validated input -> apply business rules -> database work -> response
Selected state changes -> audit insert (currently a separate commit)
```

CORS affects browser cross-origin access; it is not an authentication gate for all clients. Public endpoints do not follow the protected-request path. Authentication identifies the account, authorisation determines permitted actions, and input validation constrains data values.

2.7 Chapter Summary

The documented building blocks support the prototype, but their presence alone does not establish security. The review distinguishes a signed credential from current authorisation, input shape from business validation, and an audit table from complete traceability. The following chapters evaluate those distinctions using the project's code and test evidence.

Chapter 3: Methodology

3.1 Introduction

This chapter explains how the backend was developed and evaluated: the approach, how requirements were obtained, the architecture, tools, ethical and legal considerations, and the evaluation method.

3.2 Research / Development Approach

The project was built **iteratively**. The web application (backend and frontend) and the camera-node pipeline were two work-streams that met at one point, `POST /alerts/check-plate`. For the backend, work proceeded in slices: data model first, then authentication, then role checks, then the report lifecycle, then the plate-match endpoint and the alert and analytics routes. Each slice was tried through FastAPI's interactive `/docs` page and later covered by automated tests.

A **design-and-verify** approach was used for security: after the features worked, the code was read for weaknesses, each suspected weakness was turned into a short probe test, and findings were distinguished as observed test behaviour, configuration checks or source-code observations (§6.4).

Requirements -> Design -> Implementation -> Integration -> Testing / Security review ->
(issues found? back)

3.3 Requirements / Data Collection

Requirements came from (a) the gap in Chapter 1, (b) the natural three-way split of people who touch a stolen-vehicle report (the person who filed it, the officer investigating it, the administrator), and (c) the needs of the other components: the camera node needs one endpoint, the front end needs role-scoped listings, and the real-time member needs a hook to broadcast alerts. No stakeholder interviews or surveys are recorded, so the requirements are the group's own analysis.

The data handled by the backend is personal and sensitive: names, telephone numbers, national ID numbers, police badge numbers, vehicle details and locations. Test data used in this report is synthetic (for example `jane@test.local`); no real personal data appears in the tests.

3.4 System Architecture / Research Workflow

SVDS has three independently runnable processes that communicate only over the network: `vrrs-backend` (this report), `vrrs-frontend` and `vrrs-node`.

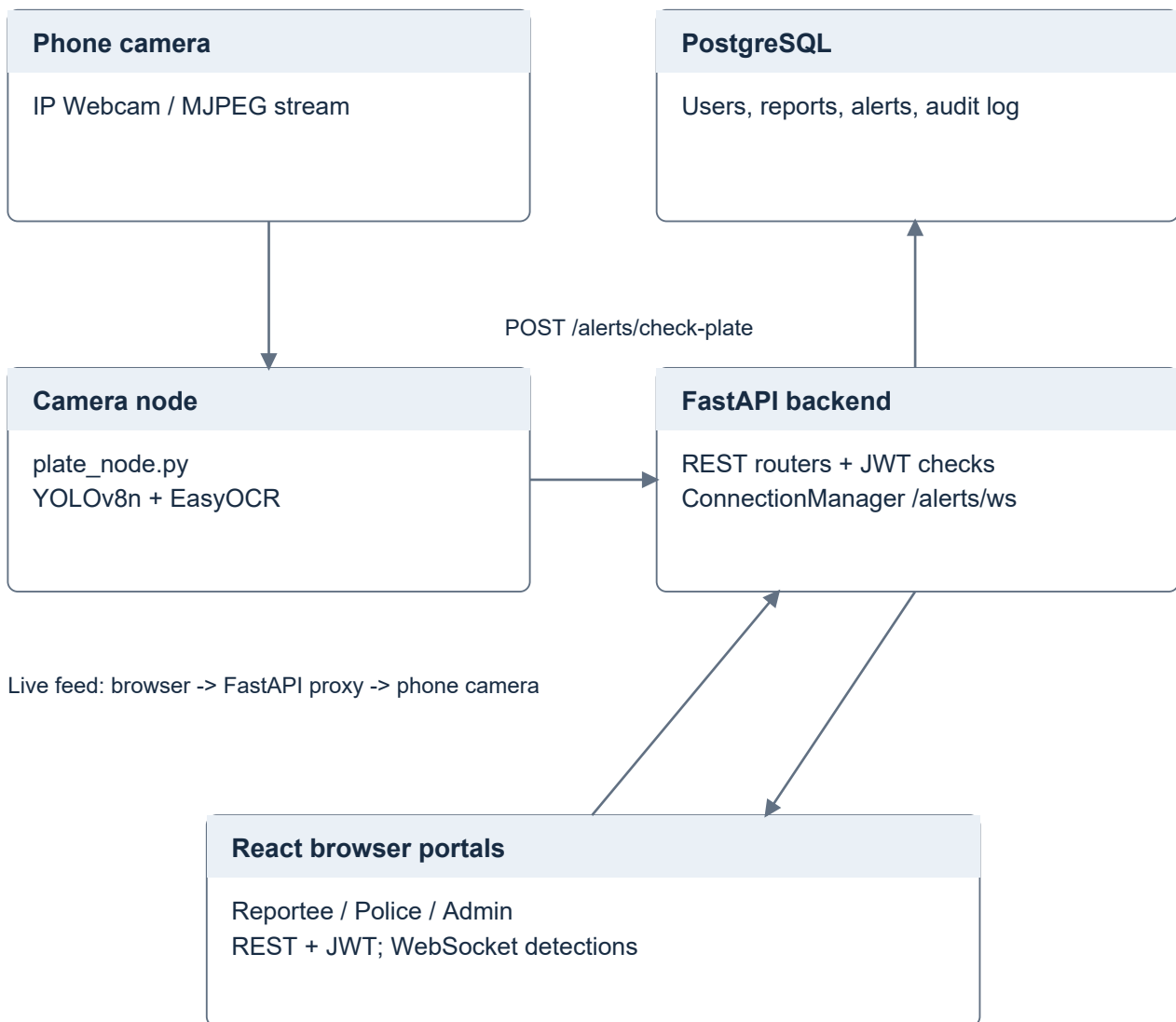


Figure 3.1: SVDS system architecture

3.5 Tools and Technologies

Table 3.1: Tools and technologies

Area	Technology
Language	Python
Web framework	FastAPI 0.111.0 with Uvicorn 0.29.0
Database	PostgreSQL, accessed through psycopg2-binary 2.9.9
ORM	SQLAlchemy 2.0.30 (Alembic 1.13.1 is listed in <code>requirements.txt</code> but not wired up)
Validation	Pydantic 2.7.1
Password hashing	passlib[bcrypt] 1.7.4 with bcrypt 4.1.3
Tokens	python-jose[cryptography] 3.3.0, algorithm HS256
Configuration	python-dotenv (.env)
Testing	pytest with FastAPI TestClient and in-memory SQLite

Area	Technology
OS	Windows 11

Versions are those pinned in `vrrs-backend/requirements.txt`.

3.6 Ethical and Legal Considerations

- **Personal data.** The system stores personal and police data. Access is limited by role, passwords are hashed, and many account and report changes are audited, with gaps and non-atomic writes described in section 6.4. The Data Protection Act, 2021 is relevant background legislation [16]; this project has not established deployment compliance or a complete retention policy.
- **Data exposure.** Reportees can read only their own reports. Police can read all reports and all users. The response models limit which user fields are returned (§4.4); the password hash is not among them.
- **Accountability.** The audit log gives a trace of who activated, resolved or deleted a report and who changed a role, but it has gaps (§6.4).
- **False matches.** A wrong plate match could point officers at an innocent driver. The backend matches reports whose stored status is missing; the generic update route can also set that status without using activation (F-3), and officers can flag alerts as false positives.
- **Unauthenticated endpoint.** `check-plate` is public because the camera has no user identity. This is a deliberate trade-off with a demonstrated consequence (§6.4).
- **Secrets.** The signing key and database password live in a `.env` file that is not committed; `.env.example` documents the keys with placeholder values.
- **Responsible testing.** Security probes were run only against the project's own code using an in-memory test database, never against a live deployment or anyone else's system.

3.7 Evaluation Method

1. **Automated tests** (pytest [11]; 31 tests at the evaluated commit, 40 after the fix) covering authentication, role access, reports, the plate-match endpoint and analytics.
2. **Targeted security probes:** short, temporary tests written to confirm or refute suspected weaknesses. They were run, their output recorded, and then removed so that the test suite stays a suite of intended behaviour.
3. **Code review** against the OWASP categories listed in §2.3.
4. **Objective-by-objective review** against B1 to B7 in §1.4.

No load or performance testing and no user evaluation were carried out.

3.8 Chapter Summary

The backend was built iteratively in slices and verified by automated tests plus probe tests that check suspected weaknesses. Data handled is sensitive; test data is synthetic.

Chapter 4: System Analysis and Design

4.1 Introduction

This chapter presents the analysis and design of the backend. Stakeholders and the architecture are shared by the group; the requirements, data design, API design and security design are those of the **backend and security component**, the subject of this report.

4.2 Stakeholder and User Analysis

Table 4.1: Stakeholder and user analysis

Stakeholder	Interest	Needs from the backend
Reportee (public)	Report a theft, follow its status	Register and log in; file and view only their own reports; edit own profile
Police officer	Investigate, activate and resolve reports, act on alerts	See all reports; activate and mark found; view and flag alerts; view analytics
Administrator	Manage accounts and oversee the system	Create police accounts, change roles, deactivate or delete users, delete reports, read the audit log
Camera node	Ask whether a plate is stolen	One public endpoint that answers CLEAR or STOLEN
Front-end and real-time developers	Build on the API	A stable, documented API; a broadcast hook for alerts

4.3 Functional Requirements

Table 4.2: Functional requirements

ID	Requirement
FR-B1	Public registration creates reportee accounts only; police and admin accounts are created by an administrator.
FR-B2	Login with email and password returns a signed token, the role and a redirect path; deactivated accounts cannot log in.
FR-B3	Passwords are stored only as bcrypt hashes; a password must be at least 6 characters.
FR-B4	Every protected route requires a valid token; role rules are checked on the server.
FR-B5	A reportee can file a report; a new report starts as <code>under_review</code> . An active (<code>missing</code>) report for the same plate blocks a new one.
FR-B6	A reportee can list and read only their own reports; police and admin can list and read all, with search and status filters and paging.
FR-B7	Police can activate a report (<code>under_review</code> -> <code>missing</code>) and mark it found (recording who and when). Police can edit report details.
FR-B8	Only an administrator can delete a report, and deleting it also deletes its alerts.

ID	Requirement
FR-B9	An administrator can create police accounts, change a user's role, deactivate or reactivate a user, and delete a user. An administrator cannot change, deactivate or delete their own account.
FR-B10	POST /alerts/check-plate normalises the plate, matches only missing reports, records an alert and triggers the real-time broadcast on a match.
FR-B11	Police can list alerts, mark them read, and flag false positives.
FR-B12	State-changing actions write an audit record naming the actor, action and target.
FR-B13	Users can view and edit their own profile and change their password (the current password must be supplied).

4.4 Non-Functional Requirements

- **Security:** no plaintext passwords; signed, expiring tokens; server-side role checks; responses limited to the fields a client needs (UserOut excludes the password hash, national ID and badge number).
- **Validation:** telephone numbers must match +260 XX XXXXXXX, national IDs NNNNNN/NN/N, enforced by Pydantic patterns at registration.
- **Consistency:** database constraints back the business rules (unique email, foreign keys, cascade delete of alerts).
- **Maintainability:** access rules are expressed once, as reusable dependencies (is_reportee, is_police, is_admin), not repeated in each route.
- **Testability:** the database session is injected so tests can swap in an in-memory database.
- **Performance:** list endpoints are paged (default 20 reports, 30 users and alerts; maximum 100).

4.5 Use Cases / User Stories

UC-B1: Register and log in. A member of the public registers with name, email, telephone, national ID and password, and receives a token and a redirect to /my. On later visits they log in with email and password. *Alternate flows:* duplicate email (409); role other than reportee requested (400); password too short (400); wrong credentials (401); deactivated account (403).

UC-B2: File a report. A logged-in reportee submits plate and vehicle details. The server upper-cases the plate, refuses it if an active report for that plate exists (409), stores it as under_review, and writes an audit record.

UC-B3: Police activate and resolve a report. An officer activates an under_review report so it becomes matchable, and later marks it found.

UC-B4: Camera checks a plate. The node sends a plate; the server returns CLEAR, or creates an alert and returns STOLEN with the vehicle's make, model and colour.

UC-B5: Administrator manages accounts. The administrator invites an officer, changes a role (a badge number is required for police or admin), or deactivates an account.

User story. *As a member of the public, I want to see only my own reports, so that other people's theft reports and personal details stay private.*

4.6 System / Solution Architecture

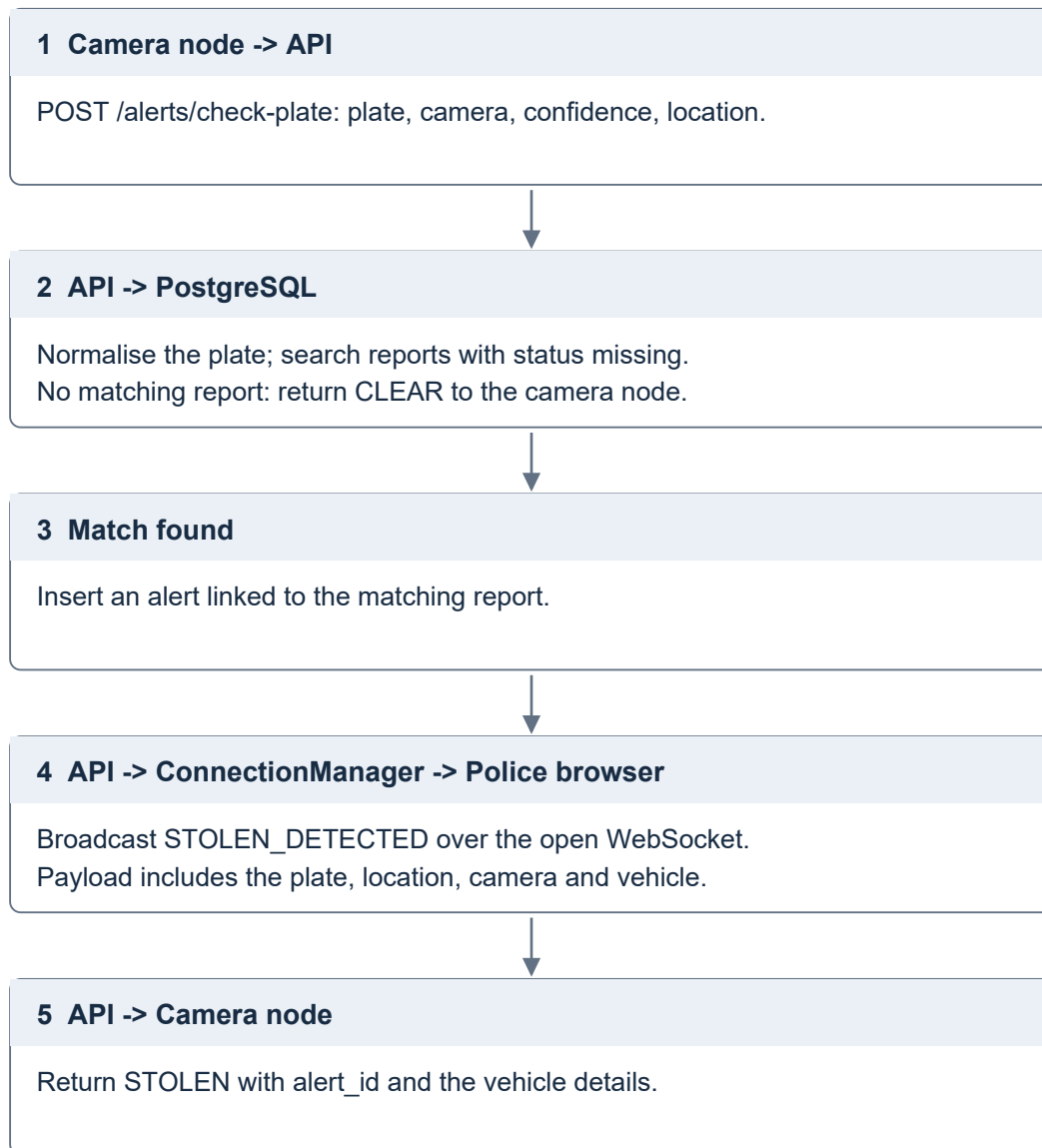


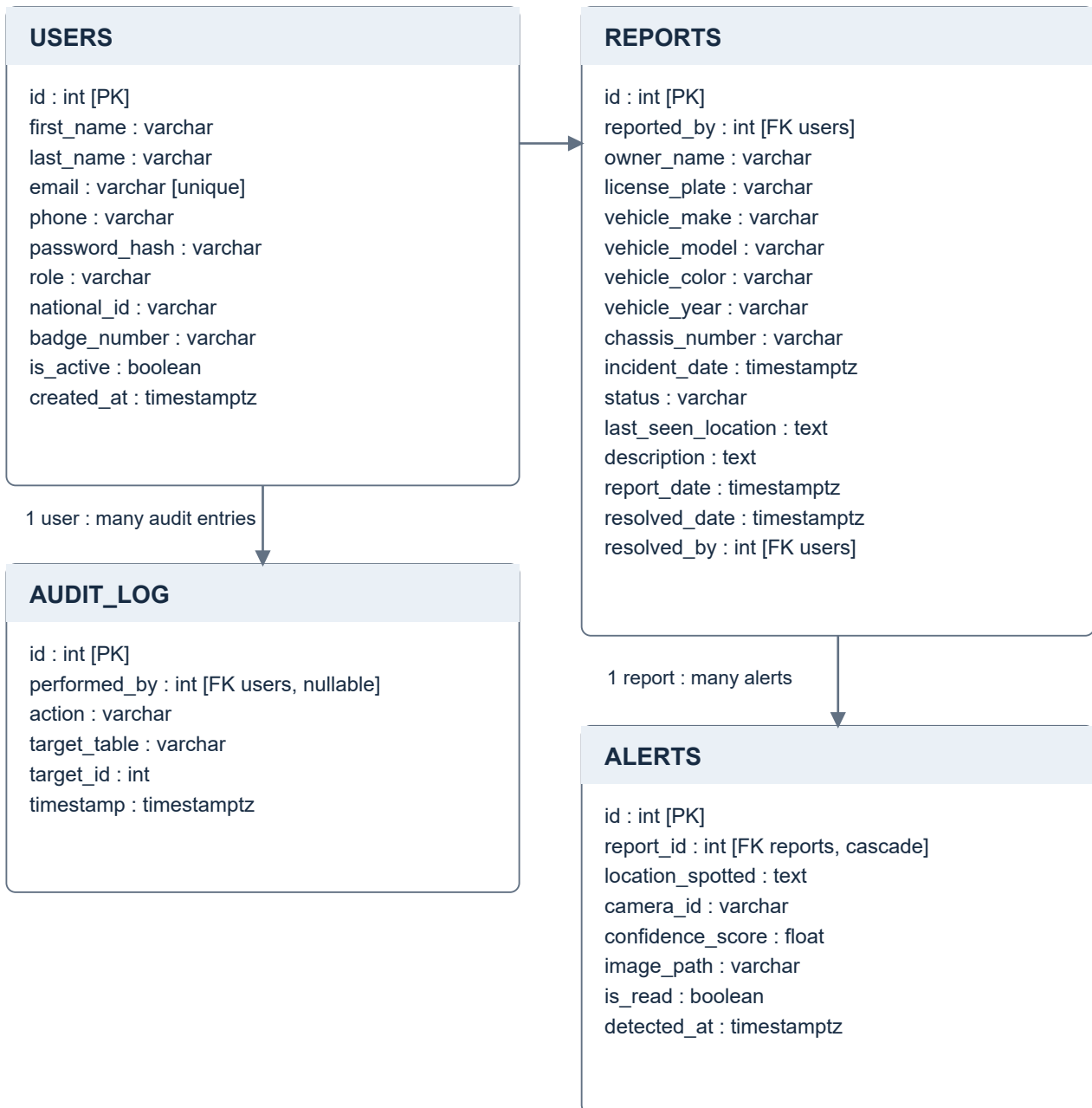
Figure 4.1: detection-to-alert sequence, which the backend's check-plate endpoint serves

The backend is a single FastAPI application. `main.py` registers six routers (auth, reports, alerts, users, analytics, system) and the CORS middleware. Shared modules provide the database session (`database.py`), the ORM models (`models.py`), the validation schemas (`schemas.py`), password and token functions (`auth.py`), and the access-control dependencies (`middleware.py`).

```
vrrs-backend/app/  
main.py      app, CORS, router registration, create_all()  
database.py  engine, session, get_db  
models.py    User, Report, Alert, AuditLog  
schemas.py   Pydantic request/response models  
auth.py      bcrypt + JWT  
middleware.py get_current_user, require_role, is_reportee/is_police/is_admin  
routers/     auth, reports, alerts, users, analytics, system
```

4.7 Database / Data Design

Four tables, defined once in `models.py`.



Users file and resolve reports (1 : many). PK = primary key; FK = foreign key.

timestampz = timestamp with time zone; timestamp = without time zone.

Figure 4.2: entity-relationship diagram

Table 4.3: Database / data design

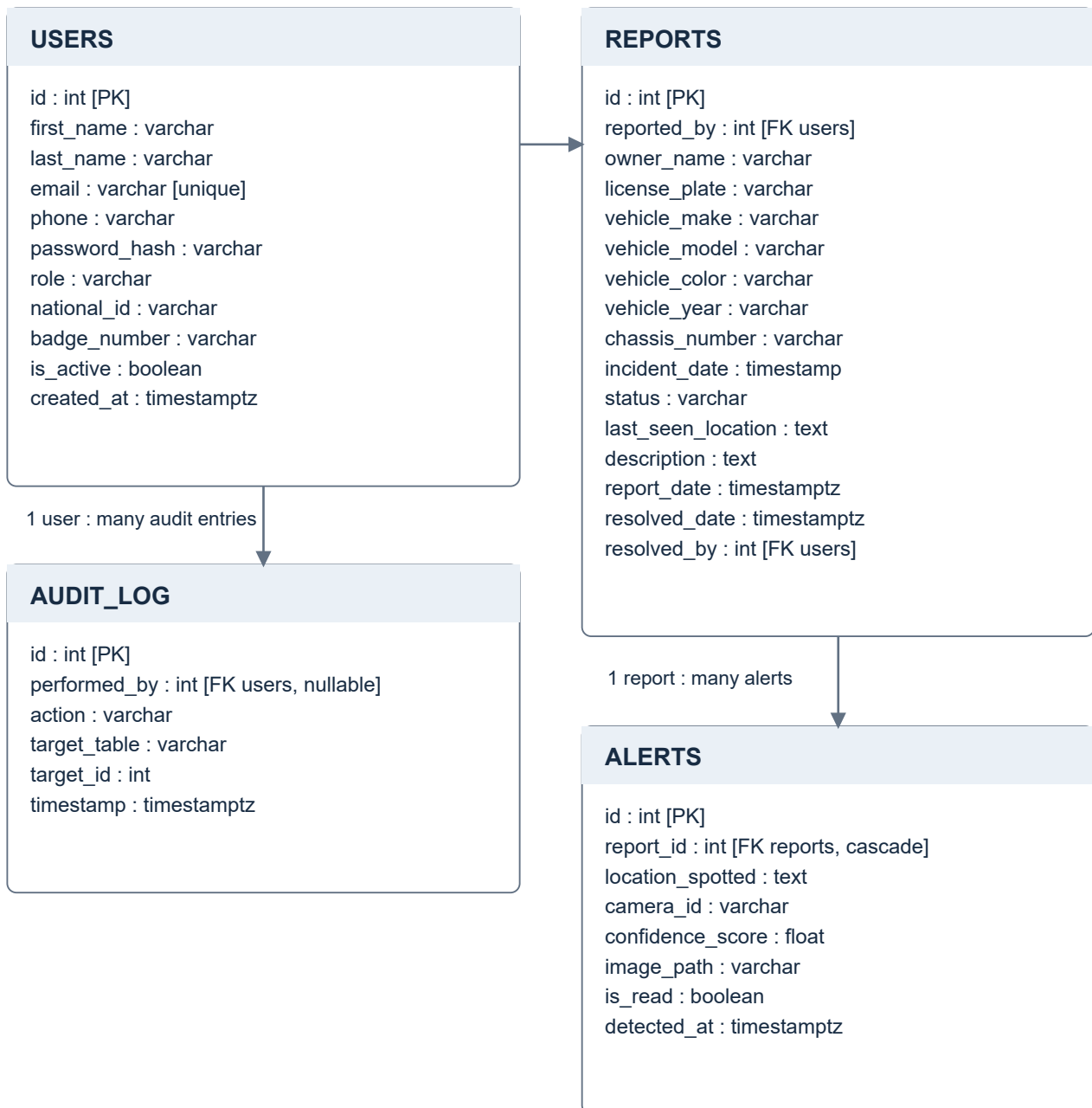
Table	Purpose	Key constraints
users	Accounts	Primary key id; unique, indexed email; role string (reportee, police, admin); is_active flag; password_hash

Table	Purpose	Key constraints
reports	Stolen-vehicle reports	Foreign key reported_by -> users; resolved_by -> users; indexed license_plate; status string
alerts	Detections	Foreign key report_id -> reports with ON DELETE CASCADE
audit_log	Action trail	Foreign key performed_by -> users (nullable)

4.7.1 The live database

The design above was checked against the running development database (PostgreSQL 18.3, database `vrrs_db`), by reading its catalogue and counting rows on 25 September 2026. Only structure and counts were read; no personal data was queried.

Figure 4.3 shows the schema as the database itself reports it (column types come from `information_schema`, not from `models.py`).



Users file and resolve reports (1 : many). PK = primary key; FK = foreign key.

timestampz = timestamp with time zone; timestamp = without time zone.

Figure 4.3: live database schema

Table 4.4: Tables in the live database

Table	Columns	Rows	Notes
users	11: id, first_name, last_name, email, phone, password_hash, role, national_id, badge_number, is_active, created_at	21	2 admin, 4 police, 15 reportee; all active
reports	16: id, reported_by, owner_name, license_plate, vehicle_make, vehicle_model, vehicle_color, vehicle_year, chassis_number, status, last_seen_location, description, report_date, resolved_date, resolved_by, incident_date	18	13 found, 3 missing, 2 under_review

Table	Columns	Rows	Notes
alerts	8: id, report_id, location_spotted, camera_id, confidence_score, image_path, is_read, detected_at	23	
audit_log	6: id, performed_by, action, target_table, target_id, timestamp	207	Largest table: it records every action

Constraints and indexes found. Four foreign keys: reports.reported_by and reports.resolved_by and audit_log.performed_by, each referencing users(id), and alerts.report_id referencing reports(id) with ON DELETE CASCADE. Every table has a primary key. users.email has a unique index; reports.license_plate has an ordinary index (the column the plate-match query searches). There is **no check constraint anywhere**, so nothing in the database stops an invalid role or status (the root of F-3).

A drift between model and database. In models.py, reports.incident_date is declared DateTime(timezone=True), but in the live database the column is timestamp without time zone, while every other timestamp column has a time zone. The most likely cause is the one already noted below: create_all() never alters an existing table, so the column kept the type it was first created with. It is small here, but it is the kind of silent difference that a migration tool such as Alembic would have caught.

Design decisions:

- **alerts.license_plate is not stored.** It is a computed property reading through the linked report, avoiding duplicated data.
- **Cascade delete.** Deleting a report also deletes its alerts (the ORM relationship uses cascade="all, delete-orphan", and the database constraint uses ON DELETE CASCADE).
- **Roles and statuses are plain strings**, not database enumerations or check constraints. This is simple, but the database itself does not stop an invalid value (§6.4).
- **No migration tool.** Base.metadata.create_all() runs at start-up. It creates missing tables but never changes existing ones, so a constraint change on an existing table needs a manual ALTER TABLE.

4.8 Interface / Interaction Design

The backend's interface is its HTTP API, documented automatically at /docs. Access is by role (yes allowed):

Table 4.5: Interface / interaction design

Route group	Endpoint(s)	Reportee	Police	Admin
Auth	POST /auth/register, /auth/login, /auth/logout	public	public	public
Reports	POST /reports/, GET /reports/, GET /reports/{id}	Yes (own only for reads)	Yes all	Yes all
Reports	PATCH /reports/{id}, /activate, /found		Yes	Yes

Route group	Endpoint(s)	Reportee	Police	Admin
Reports	DELETE /reports/{id}			Yes
Alerts	POST /alerts/check-plate	public	public	public
Alerts	GET /alerts/, /{id}, /unread/count, PATCH .../read, /read-all, /false-positive		Yes	Yes
Users	GET/PATCH /users/me, PATCH /users/me/password	Yes	Yes	Yes
Users	GET /users/, GET /users/{id}		Yes	Yes
Users	POST /users/invite, PATCH /users/{id}/role, /toggle-active, DELETE /users/{id}			Yes
Analytics	GET /analytics/...		Yes	Yes (user-growth admin only)
System	GET /system/audit			Yes
System	GET /system/camera-nodes, /system/live-feed		Yes	Yes
System	GET /system/health	public	public	public

/system/health currently has no authentication dependency. Its active_sessions metric counts active user accounts, not live authenticated sessions; the service-status labels are not independent checks of every component. Restricting detailed operational metrics should be considered before deployment. This is an additional scope observation, not a newly measured result in the original F-1 to F-10 series.

The interactive documentation FastAPI generates from these routes is shown in Figure 4.4; it lists every route above with its request and response schemas.

SVDS — Vehicle Registry and Reporting System 1.0.0 OAS 3.1

/openapi.json

Zambia Police Service Vehicle Reporting API

Authorize 

Authentication

POST /auth/register Register

POST /auth/login Login

POST /auth/logout Logout

Reports

POST /reports/ Create Report

GET /reports/ Get Reports

GET /reports/{report_id} Get Report

PATCH /reports/{report_id} Update Report

DELETE /reports/{report_id} Delete Report

PATCH /reports/{report_id}/activate Activate Report

PATCH /reports/{report_id}/found Mark Found

Figure 4.4: the generated API documentation at /docs - detail panel 1 of 4

Alerts

POST	/alerts/check-plate	Check Plate
GET	/alerts/unread/count	Unread Count
PATCH	/alerts/read-all	Mark All Read
GET	/alerts/	Get Alerts
GET	/alerts/{alert_id}	Get Alert
PATCH	/alerts/{alert_id}/read	Mark Read
PATCH	/alerts/{alert_id}/false-positive	Flag False Positive

Users

POST	/users/invite	Invite Police Officer
GET	/users/me	Get My Profile
PATCH	/users/me	Update My Profile
PATCH	/users/me/password	Change Password
GET	/users/	Get All Users
GET	/users/{user_id}	Get User
DELETE	/users/{user_id}	Delete User
PATCH	/users/{user_id}/role	Change Role
PATCH	/users/{user_id}/toggle-active	Toggle Active

Figure 4.4: the generated API documentation at /docs - detail panel 2 of 4

Analytics

GET	/analytics/summary	Get Summary
GET	/analytics/reports-per-month	Reports Per Month
GET	/analytics/status-breakdown	Status Breakdown
GET	/analytics/detections-per-node	Detections Per Node
GET	/analytics/alerts-over-time	Alerts Over Time
GET	/analytics/recovery-time	Avg Recovery Time
GET	/analytics/user-growth	User Growth

System

GET	/system/audit	Get Audit Log
GET	/system/camera-nodes	Get Camera Nodes
GET	/system/live-feed	Get Live Feed
GET	/system/health	Get System Health

default

GET	/	Root
-----	---	------

Figure 4.4: the generated API documentation at /docs - detail panel 3 of 4

Schemas		
AlertCreate >	Expand all	object
AlertOut >	Expand all	object
AuditLogOut >	Expand all	object
AuthResponse >	Expand all	object
HTTPValidationError >	Expand all	object
PasswordChange >	Expand all	object
ReportCreate >	Expand all	object
ReportOut >	Expand all	object
ReportUpdate >	Expand all	object
RoleChange >	Expand all	object
UserInvite >	Expand all	object
UserLogin >	Expand all	object
UserOut >	Expand all	object
UserRegister >	Expand all	object
UserUpdate >	Expand all	object
ValidationError >	Expand all	object

Figure 4.4: the generated API documentation at /docs - detail panel 4 of 4

The web pages that use these routes were designed by the frontend member.

4.9 Algorithms / Models / Technical Design

4.9.1 Authentication and token design



Figure 4.5: authentication sequence

On login the server looks up the user by lower-cased email, verifies the password against the stored bcrypt hash, rejects deactivated accounts, writes an audit record, and returns a JWT whose claims are `sub` (user id), `role` and `exp` (default 24 hours, configurable). The token is signed with HS256 using `SECRET_KEY` from the environment. On each request, `get_current_user` decodes the token, then **looks the user up in the database** and returns the id and role from that row; a user who no longer exists or is not active is refused with 401. (At the first evaluated commit, a349fc6, it returned the id and role **taken from the token** and did not consult the database; that was findings F-1 and F-2, fixed in b326b21. §5.4.2 shows the code.)

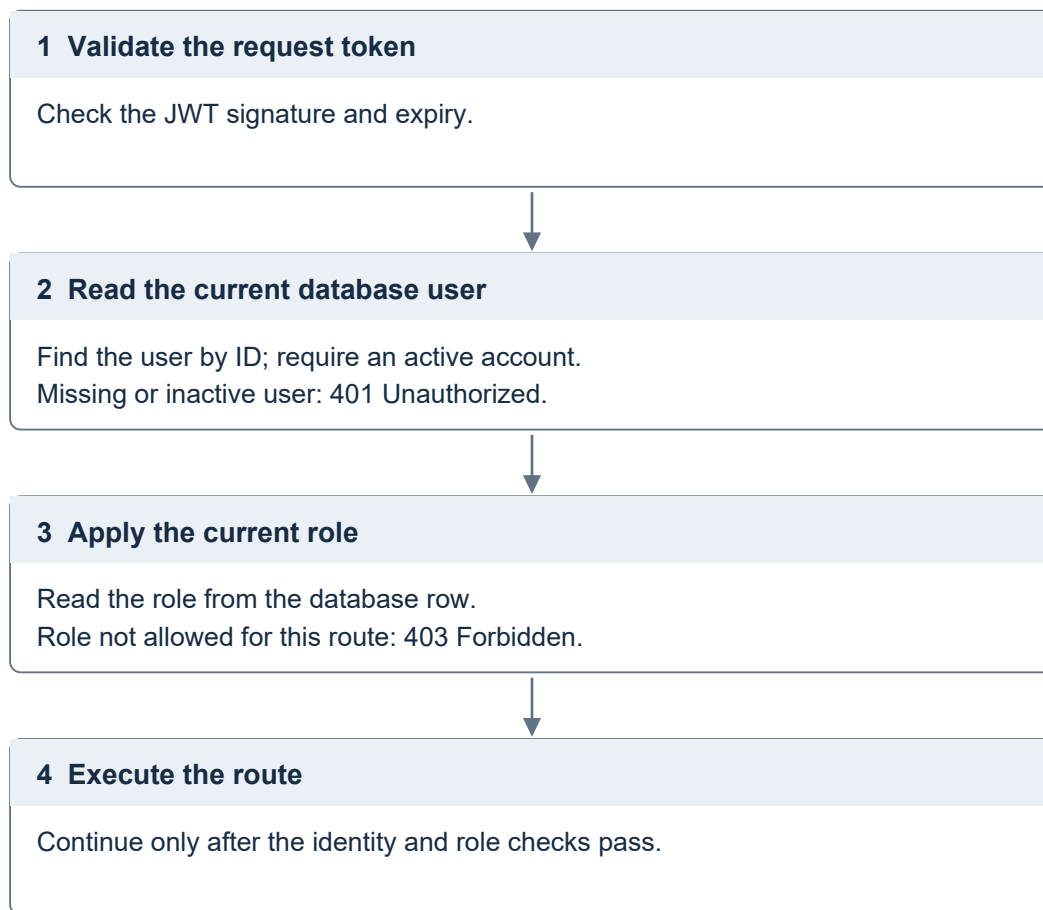
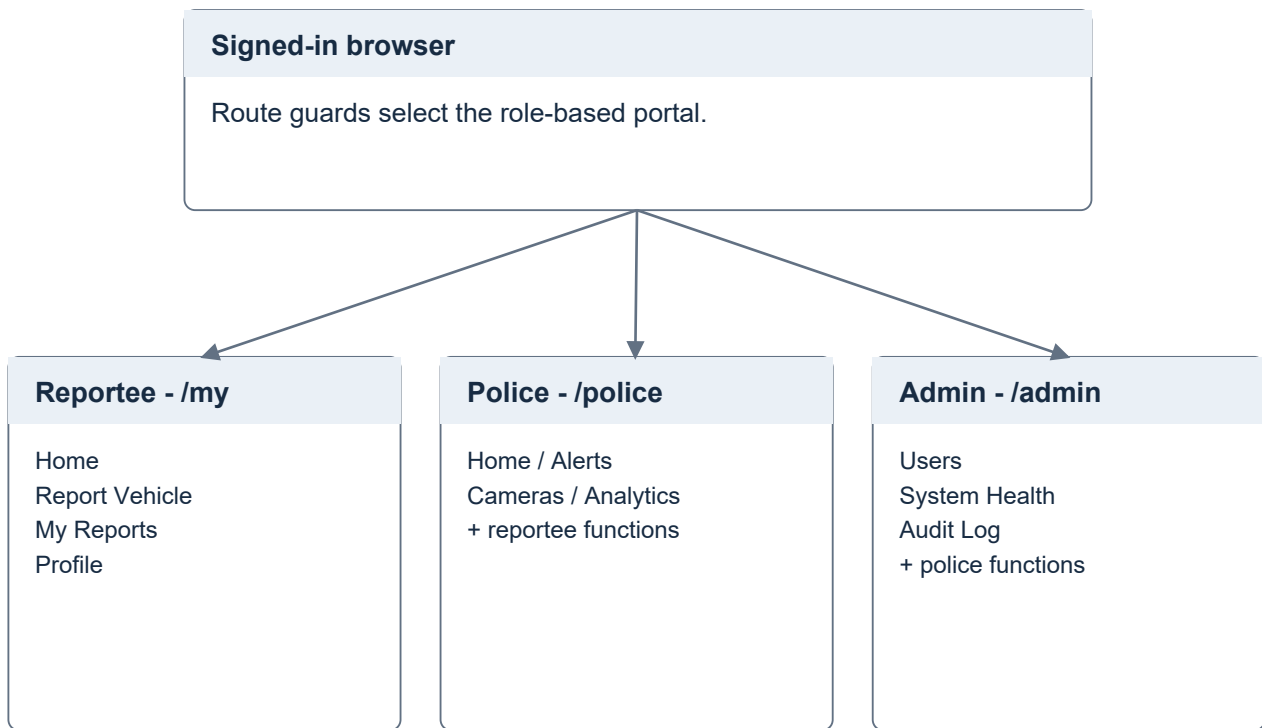


Figure 4.6: per-request authorisation after the fix

4.9.2 Role enforcement

`require_role(*roles)` builds a dependency that returns 403 if the current database role is not in the list. Three shortcuts wrap it: `is_reportee` (all three roles), `is_police` (police and admin), `is_admin` (admin only). Routes attach them with `Depends(...)`. Row-level rules are added inside the route: a reportee's list is filtered by `reported_by`, and reading another reportee's report returns 403.



The API independently checks the current database user and permitted role.

Figure 4.7: role-based route access

4.9.3 Plate matching

check-plate removes every character except letters and digits from the received plate, upper-cases it, and compares it with the same normalisation applied to the stored plate in SQL (`regexp_replace(upper(license_plate), '^[A-Z0-9]', '', 'g')`). It matches only reports whose status is missing. This makes abc 1234, ABC-1234 and ABC1234 equal. On a match it inserts an alert, calls the WebSocket broadcast (the real-time member's component) and returns `STOLEN` with make, model and colour.

4.9.4 Report lifecycle

```
under_review --(police: activate)--> missing --(police: mark found)--> found
```

Only missing reports are matched. activate accepts only under_review; found refuses an already-found report but does not require the previous status to be missing, so it also permits direct resolution from under_review. The general `PATCH /reports/{id}` route is separate and is discussed in §6.4.

4.10 Chapter Summary

The backend is a FastAPI service over four PostgreSQL tables. Access control is expressed as three reusable role dependencies plus per-row ownership rules. Authentication uses bcrypt and a signed JWT that carries the role. The plate-match endpoint normalises both sides so formatting differences do not prevent a match.

Chapter 5: Implementation / Development

5.1 Introduction

This chapter describes how the backend was built: its modules, the key implementation decisions, how it connects to the other components, the problems met, and the student's own contribution.

5.2 Development Environment

Table 5.1: Development environment

Item	Detail
Machine	Windows 11 PC
Language and runtime	Python (the test virtual environment used Python 3.12.10)
Database	PostgreSQL (vrrs_db), connection string from DATABASE_URL
Frameworks	FastAPI 0.111.0, SQLAlchemy 2.0.30, Pydantic 2.7.1
Security libraries	passlib 1.7.4 + bcrypt 4.1.3, python-jose 3.3.0
Tools	Swagger UI at /docs for manual trials, pytest for automated tests, Git
Configuration	.env (not committed); .env.example lists DATABASE_URL, SECRET_KEY, ALGORITHM, TOKEN_EXPIRY_HOURS, CORS_ORIGINS

5.3 System Components / Modules

Table 5.2: System components / modules

Module	Lines	Purpose	Built by
app/main.py	55	App creation, CORS, router registration, schema creation	Backend (this report)
app/database.py	20	Engine, session factory, get_db dependency	Backend (this report)
app/models.py	65	User, Report, Alert, AuditLog ORM models	Backend (this report)
app/schemas.py	132	Pydantic request and response models with validation patterns	Backend (this report)
app/auth.py	33	bcrypt hashing; JWT create and decode	Backend (this report)
app/middleware.py	34	get_current_user, require_role, role shortcuts	Backend (this report)
app/routers/auth.py	48	Register, login, logout	Backend (this report)
app/routers/users.py	125	Profile, password change, admin user management	Backend (this report)
app/routers/reports.py	94	Report create, list, read, update, activate, found, delete	Backend (this report)

Module	Lines	Purpose	Built by
app/routers/alerts.py	123	check-plate and alert listing (this report); WebSocket manager and endpoint (real-time member)	Shared
app/routers/analytics.py, system.py	87, 146	Statistics; audit-log listing, camera status, feed proxy, health	Frontend and real-time members
tests/	5 test files and conf test.py	Automated tests	See §5.7

Line counts are historical measurements from a349fc6; they are not current counts after the regression fixes.

5.4 Key Implementation Details

5.4.1 Password hashing and tokens (auth.py)

```
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password): return pwd_context.hash(password)
def verify_password(plain, hashed): return pwd_context.verify(plain, hashed)

def create_access_token(user_id, role):
    payload = {"sub": str(user_id), "role": role,
               "exp": datetime.utcnow() + timedelta(hours=TOKEN_EXPIRY_HOURS)}
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)
```

decode_token returns None on any signature or expiry error. The signing key is read from the environment. The code falls back to the string "changeme" if SECRET_KEY is unset; on the development machine the key is set and is not the default (checked, §6.4).

5.4.2 Access control (middleware.py)

Protected HTTP routes use get_current_user directly or through role dependencies. The WebSocket and live-feed routes call the shared resolve_token_user helper. Since commit b326b21 it re-reads the user from the database on each request (condensed from the source):

```
def resolve_token_user(token: str, db: Session):
    payload = decode_token(token) if token else None
    try:
        user_id = int(payload["sub"]) if payload else None
    except (KeyError, ValueError, TypeError):
        user_id = None
    user = db.query(User).filter(User.id == user_id).first() if user_id is not None else None
    if not user or not user.is_active:
        raise HTTPException(status_code=401, detail="Invalid or expired token.",
                             headers={"WWW-Authenticate": "Bearer"})
    return {"id": user.id, "role": user.role} # role from the database, not the token

def get_current_user(token: str = Depends(oauth2_scheme), db: Session = Depends(get_db)):
    return resolve_token_user(token, db)
```

Before this change get_current_user returned {"id": int(payload["sub"]), "role": payload["role"]} straight from the token, which is why a deactivated or demoted user kept their access (F-1, F-2). The cost is one primary-key query per request. The same function now also guards the WebSocket handshake and the live-feed proxy, and the WebSocket re-checks it on every heartbeat and before

every broadcast, so a revoked officer stops receiving alerts. Those two endpoints belong to the real-time member's work; the shared check is noted here because it lives in this component.

```
def require_role(*roles):
    def checker(user: dict = Depends(get_current_user)):
        if user["role"] not in roles:
            raise HTTPException(
                status_code=403, detail=f"Access denied. Required roles: {list(roles)}"
            )
        return user
    return checker

is_reportee = require_role("reportee", "police", "admin")
is_police = require_role("police", "admin")
is_admin = require_role("admin")
```

(In the source these are small wrapper functions; they are shown here in condensed form.) A route needs one line to require a role, for example `user=Depends(is_admin)`.

5.4.3 Ownership and lifecycle rules (reports.py)

- `create_report` upper-cases and trims the plate, returns 409 if a missing report with that plate exists, saves the report as `under_review`, and writes an audit row.
- `get_reports` filters by `reported_by` when the caller is a reportee, and supports status, search (plate or owner name, case-insensitive), skip and limit.
- `get_report` returns 403 if a reportee asks for someone else's report.
- `activate_report` accepts only `under_review`; mark found records `resolved_by` and `resolved_date`.
- `delete_report` is admin-only and relies on the cascade to remove alerts.

5.4.4 Plate matching (alerts.py)

```
normalized = re.sub(r"^[A-Z0-9]", "", data.license_plate.upper())
report = db.query(Report).filter(
    func.regexp_replace(func.upper(Report.license_plate), r"^[A-Z0-9]", "", "g") == normalized,
    Report.status == "missing",
).first()
if not report:
    return {"status": "CLEAR"}
new_alert = Alert(
    report_id=report.id, location_spotted=data.location_spotted,
    camera_id=data.camera_id, confidence_score=data.confidence_score,
    image_path=data.image_path,
)
db.add(new_alert); db.commit(); db.refresh(new_alert)
# Selected fields shown; the full broadcast also carries vehicle details.
await manager.broadcast({
    "type": "STOLEN_DETECTED", "alert_id": new_alert.id,
    "plate": data.license_plate.upper(),
}, db)
```

5.4.5 Admin safeguards (users.py)

An administrator cannot change, deactivate or delete their own account (prevents locking out the last admin by accident). Assigning police or admin requires the target user to have a badge number. Password change requires the current password and rejects an unchanged or too-short new password. `UserUpdate`, the model used for profile edits, has no `role` or `is_active` field, so a user cannot promote themselves through their profile.

5.4.6 Audit records

The live database holds 207 audit rows against 21 users (25 September 2026). Registration, login, report filing, updates, activation, found, deletion, account creation, role change, activation or deactivation of an account, password change, profile update and false-positive flags each add an `AuditLog` row with the actor's id, a text description, the target table and the target id.

5.5 Integration

- **Camera node (AI/vision member):** the only contract is `POST /alerts/check-plate` with `license_plate`, `camera_id`, `confidence_score`, `location_spotted` and optional `image_path`. The backend normalises the plate itself, so the node does not need to reproduce the matching rules.
- **Real-time member:** `check-plate` calls the `ConnectionManager.broadcast` method they wrote; the `WebSocket` handshake reuses the same `resolve_token_user` database check and admits only police and admin users.
- **Frontend member:** the front end calls the routes in §4.8 with the token in an `Authorization: Bearer` header and reads the role from the login response. CORS reads `CORS_ORIGINS`; if the variable is absent, the defaults are `localhost:5173` and `5174` plus their `127.0.0.1` equivalents.

`get_allowed_origins()` splits and trims the configured comma-separated origins. An explicitly empty string produces an empty allow-list. CORS governs browser cross-origin access; it does not authenticate camera nodes or prevent requests from non-browser clients [1].

5.6 Challenges and Solutions

Table 5.3: Challenges and solutions

Challenge	Cause	Solution
Deleting a report that had detections failed with a database error	The <code>alerts.report_id</code> foreign key was created with no delete action	Added <code>ondelete="CASCADE"</code> to the foreign key and <code>cascade="all, delete-orphan"</code> to the relationship. Because <code>create_all()</code> does not alter existing tables, the live constraint was also altered by hand and the change verified in PostgreSQL's constraint catalogue. Covered by a regression test
A PostgreSQL-only function in <code>check-plate</code> could not run under the test database	SQLite has no four-argument <code>regexp_replace</code>	The test set-up registers a Python implementation on each SQLite connection, so the production query runs unchanged in tests
Cascade delete did not fire in tests	SQLite ignores foreign keys unless enabled	The test set-up runs <code>PRAGMA foreign_keys=ON</code> per connection
Tests must not touch the real database	<code>database.py</code> connects to PostgreSQL at import	The tests override the <code>get_db</code> dependency with an in-memory SQLite session using a static pool
Camera cannot log in	It has no user identity	<code>check-plate</code> is left public and limited in what it can do (it cannot create or alter reports); see §6.4 for the consequence

Challenge	Cause	Solution
Roles enforced only in the UI would be bypassable	Clients can send any request	Every route declares its role dependency; a parametrised test hits police and admin routes as a reportee

5.7 Individual Contribution (group project)

The student was responsible for **work areas 3 and 4** of the group's seven (Appendix A): the backend API and database, and security and access control. This covers:

- the four-table data model and its relationships, keys and cascade;
- the FastAPI application structure and configuration;
- registration, login, password hashing and token issue and checking;
- the role dependencies and the ownership rules on reports;
- user management, including the admin safeguards;
- the report lifecycle endpoints and the plate-match endpoint;
- writing of audit records;
- the security review in Chapter 6.

Work by other members that this component uses, **not claimed here**: the WebSocket manager and endpoint, camera status, live-feed proxy and system health (real-time and integration member); analytics endpoints and the React pages (frontend member); the detection model and camera node (AI/vision member).

Tests. The group's role split placed "tests" with the frontend member. The backend tests (31 at the evaluated commit, 40 now) exercise this component, and the student ran them and interpreted their results for this report. **Who wrote each test file should be confirmed by the student before submission**; §6.3 describes them without claiming authorship.

5.8 Chapter Summary

At the initial evaluation, the backend comprised about 730 lines of Python across ten modules (`main.py`, `database.py`, `models.py`, `schemas.py`, `auth.py`, `middleware.py` and four routers; the generic `model.py` placeholder is not counted). It hashes passwords with `bcrypt`, issues signed role-carrying tokens, enforces roles with reusable dependencies, and answers plate-match queries. Its notable implementation problems were the missing cascade and the gap between PostgreSQL and the SQLite test database.

Chapter 6: Testing, Results and Evaluation

6.1 Introduction

This chapter evaluates the backend. It reports the automated test results, then the security findings from targeted probes, then judges each individual objective. The retained automated tests can be re-run. Historical probe results are transcribed in Appendix D, but the deleted probe source prevents exact independent reproduction of that earlier run.

6.1.1 Version of the code evaluated

Note on the version of the code evaluated. The findings F-1 to F-10 were first obtained against the code as committed at commit a349fc6 (31 tests; line counts and coverage figures in this report are from that commit). The code was then changed and committed as b326b21 (middleware.py, alerts.py, system.py, main.py, conftest.py, a new tests/test_regressions.py, and two front-end files with a new front-end test). Against that commit the backend suite has **40 tests, all passing** (thesis/assets/test-results/backend-pytest-output.txt), and the front-end suite has one test, which also passes. The original probes were then **re-run** against the fixed code (Table 6.4 in §6.4.1).

Fixed and re-verified: **F-1** and **F-2** (every protected request now re-reads the user from the database, so a deactivated, demoted or deleted user's old token is rejected with 401 or 403, and the role comes from the database, not the token) and **F-9** (CORS_ORIGINS is now read). The same check now also protects the WebSocket and the live-feed proxy. *Still open, re-confirmed by re-running the probes:* F-3, F-4, F-6 and F-7. *Still open, not re-probed:* F-5 (TC-19 still passes), F-8 (the fallback is unchanged) and F-10 (the script is unchanged).

6.2 Testing Strategy

Table 6.1: Testing strategy

Level	Technique	Target
Integration (API)	pytest with FastAPI TestClient, real routes, real ORM	Auth, RBAC, reports, plate matching, analytics
Security probing	Short temporary tests confirming suspected weaknesses	Token lifetime, status validation, public endpoint, login attempts, signing key, logout
Code review	Reading the source against OWASP categories	All backend modules
Acceptance / user testing	None	See §6.5

Test environment. The tests use an in-memory SQLite database, not PostgreSQL, and the real get_db dependency is replaced by an override. Two SQLite gaps are closed in conftest.py: foreign keys are switched on, and a Python version of PostgreSQL's four-argument regexp_replace is registered so the production matching query runs unchanged. This means results say nothing about behaviour that differs between SQLite and PostgreSQL (for example locking, data types or the constraint catalogue).

6.3 Test Cases and Results

The suite has **40 tests, all passing** (re-run against commit b326b21 while preparing this report; the captured output is in `thesis/assets/test-results/backend-pytest-output.txt`). Thirty-one of them are the tests that existed at the evaluated commit a349fc6 and are described below. The other nine were added with the fix and are summarised in the last row of the table.

Table 6.2: Automated test cases

ID	Area	Test case	Expected result	Status
TC-01	Auth	Register a reportee	201; role reportee; redirect /my	Pass
TC-02	Auth	Register with an existing email	Rejected	Pass
TC-03	Auth	Register asking for a non-reportee role	Rejected	Pass
TC-04	Auth	Register with a short password	Rejected	Pass
TC-05	Auth	Login with correct credentials	Token returned	Pass
TC-06	Auth	Login with a wrong password	401	Pass
TC-07	Auth	Login to a deactivated account	Rejected	Pass
TC-08	Auth	Protected route without a token	401	Pass
TC-09 to TC-12	RBAC	Reportee requests /users/, /system/audit, /alerts/, /analytics/summary	403 each	Pass (4 cases)
TC-13, TC-14	RBAC	Police request /system/audit, /analytics/user-growth	403 each	Pass (2 cases)
TC-15	RBAC	Police request /alerts/	200	Pass
TC-16	RBAC	Admin request /system/audit	200	Pass
TC-17	RBAC	Reportee tries to delete a report	403	Pass
TC-18	Reports	Reportee files a report	201; plate upper-cased; under_review	Pass
TC-19	Reports	A second report while the first is only under_review	201 (allowed; see §6.4 finding F-5)	Pass
TC-20	Reports	Filing against a plate with a missing report	409	Pass
TC-21	Reports	Reportee lists reports	Sees only their own	Pass
TC-22	Reports	Police activate then mark found	missing then found; resolved_by set	Pass
TC-23	Reports	Admin deletes a report that has an alert	204; report and alert both gone	Pass
TC-24	Plate match	Unknown plate	CLEAR	Pass
TC-25	Plate match	Exact plate on a missing report	STOLEN	Pass

ID	Area	Test case	Expected result	Status
TC-26	Plate match	Different case and spacing	STOLEN	Pass
TC-27	Plate match	Plate on a found report	CLEAR	Pass
TC-28	Alerts	Unread count and marking read	Counts update	Pass
TC-29	Alerts	Flag false positive	Audit row written	Pass
TC-30, TC-31	Analytics	Confidence averages across mixed scales	Normalised	Pass (owned by the analytics work; listed for completeness)
TC-32 to TC-40	Regression (test s/test_regres sions.py)	A deactivated, demoted or deleted user's token is refused on a normal route, on the live-feed proxy and on the WebSocket; the WebSocket re-checks on each heartbeat and before each broadcast; configured and default CORS origins; health metrics on SQLite	401/403 or WebSocket close code 1008 as appropriate; CORS as configured; 200 with null pool figures	Pass

A consistency run on 25 September 2026 at checkout bea7410 again produced **40 passed, 87 warnings in 23.19 seconds**, using Python 3.12.10, pytest 9.1.1, FastAPI 0.111.0, Pydantic 2.7.1, SQLAlchemy 2.0.30 and HTTPX 0.28.1. The warnings concern deprecated APIs; they are not test failures. The frontend heartbeat suite also passed its one test. The backend application/tests and frontend files have no diff between b326b21 and this checkout. A verification summary is retained in thesis/assets/test-results/report-consistency-check-2026-09-25.txt.

Counting the parametrised cases, TC-01 to TC-31 are 31 tests and TC-32 to TC-40 are nine more. TC-19 is worth reading closely: it is a test that *documents* current behaviour, and that behaviour is itself a weakness (F-5).

6.4 Security Findings from Probes

The retained historical output records probes for F-1 to F-4, F-6 and F-7, a signing-key check relevant to F-8, and an invalid registration probe. The temporary probe source is not retained, so its original test-file count and exact execution cannot be independently reconstructed from the current repository. F-5 is supported by retained test TC-19. The initial observations below must be read together with the post-fix status in Table 6.4.

Table 6.3: Findings

ID	Suspected weakness	What was observed	Severity (student's judgement)
F-1	A token stays valid after the user is deactivated	After setting <code>is_active = False</code> , the user's existing token still got 200 from GET <code>/users/me</code> , while a fresh login was refused with 403	High
F-2	A token keeps the role it was issued with	After an admin was demoted to reportee, their old token still got 200 from GET <code>/system/audit</code> (an admin-only route)	High

ID	Suspected weakness	What was observed	Severity (student's judgement)
F-3	PATCH /reports/{id} accepts any status	A police user set status to "banana"; the server returned 200 and stored it. The same route can also set missing without going through activate	Medium
F-4	check-plate is unauthenticated and not rate-limited	Three identical unauthenticated calls for one active plate returned STOLEN three times and created three alert rows	Medium
F-5	The duplicate-report check ignores under_review reports	A second report for the same plate was accepted while the first was under_review (TC-19)	Low
F-6	No limit on failed logins	Twenty wrong-password attempts against one account all returned 401; none was locked out or slowed	Medium
F-7	Logout does not invalidate the token	After POST /auth/logout, the same token still got 200 from GET /users/me	Medium

Three further observations came from reading and from checking configuration:

- **F-8 Signing key.** The code falls back to "changeme" if SECRET_KEY is unset. A check showed the development configuration uses a 47-character key that is not the default, and a token forged with "changeme" was rejected. The weakness is therefore *latent* (a deployment that forgets to set the variable would be forgeable), not active.
- **F-10 Admin seed script.** vrrs-backend/create_admin.py, the script that creates the first administrator, falls back to a hard-coded default password if ADMIN_PASSWORD is not set, prints the credentials to the console, and is committed to the repository. Anyone who runs it without setting the variable creates an administrator with a password that is public in the source. It also defaults DATABASE_URL to a local SQLite file if none is set. This was found by reading the script; it was not run.
- **F-9 CORS configuration (initial version; fixed).** At a349fc6, the configured value was ignored in favour of hard-coded localhost origins. Since b326b21, get_allowed_origins reads a comma-separated value, trims whitespace and removes empty entries. An explicitly empty value allows no browser origin; only an absent variable enables the four local-development defaults. The retained CORS tests cover these cases.

A probe that was invalid. A probe intended to test whether a six-character password such as abcdef is accepted at registration used an incomplete payload (missing the required national ID) and returned 422 for that reason. It shows nothing about the password rule, so **no finding is claimed**. The password minimum (six characters, enforced in the route) is covered by TC-04. The six-character minimum is a code observation. NIST SP 800-63B-4 recommends at least 15 characters for single-factor passwords, blocking common or compromised passwords, and limiting failed attempts; it does not recommend mandatory character-type composition rules [15]. The absence of a composition rule is therefore not itself a defect.

Additional points from reading the code (not probed):

- Roles and statuses are free-text columns without database check constraints. `ReportUpdate.status` accepts an arbitrary string (F-3). In contrast, public registration restricts the role to `reportee`, and the admin role-change route checks an explicit allow-list. A plain-string schema alone does not mean the role-changing API accepts arbitrary roles.
- `email` is a plain string, not validated as an email address.
- `UserUpdate` has no format pattern for phone or national ID, unlike registration, so profile edits can store malformed values.
- Failed logins are not written to the audit log; successful ones are.
- Many list and detail routes are `async` functions that call blocking database code; under load this could stall the event loop. This was not measured.
- In `delete_report`, the audit row is written *after* the delete is committed, so a failure between the two would leave a deletion with no record.

6.4.1 Re-run of the probes after the fix

The probes were re-written as temporary tests and run against commit `b326b21` on the in-memory database (same method as above; the temporary file was deleted afterwards). This checks the fix against the exact scenarios that exposed the weaknesses, not just against the new regression tests.

Table 6.4: Finding status at commit `b326b21`

ID	Result before (a349fc6)	Result after (b326b21)	Status
F-1	Deactivated user's token: 200 on <code>/users/me</code>	401	Fixed
F-2	Demoted admin's old token: 200 on <code>/system/audit</code>	403	Fixed
F-3	PATCH status banana: 200; missing via PATCH: 200	Same: 200 and 200	Open
F-4	Three unauthenticated check-plate calls: three STOLEN, three alert rows	Same: alert ids 1, 2, 3	Open
F-5	Second report accepted while first under_review (TC-19)	TC-19 still passes	Open
F-6	Twenty wrong passwords: all 401	Same: {401} for all twenty	Open
F-7	Token after POST <code>/auth/logout</code> : 200	Same: 200	Open
F-8	Default key changeme is a latent fallback	Code unchanged	Open (latent)
F-9	CORS_ORIGINS ignored	Read from the environment; TC-32 to TC-40 cover it	Fixed
F-10	<code>create_admin.py</code> default password	Script unchanged (by reading)	Open

Three of the ten findings are fixed, including the two rated High. One consequence of the fix is worth stating: because F-1 and F-2 were fixed by re-checking the database, a logged-out token (F-7) is still accepted, since logout changes nothing in the database. F-7 therefore needs a revocation mechanism

such as a token deny-list or server-side session version. Shorter token lifetimes only reduce the exposure window; they do not provide immediate logout invalidation.

One behaviour change from the fix should be noted: `/system/live-feed` now answers 403 (not 401) for a valid token whose user is not police or admin, and an empty `CORS_ORIGINS` value now allows no origin at all.

6.4.2 Coverage and remaining gaps

The nine added backend cases cover account deactivation, demotion and deletion on HTTP routes, the live-feed authorisation gate and WebSocket connection; heartbeat and broadcast rechecks; configured/default CORS; and SQLite-compatible health metrics. One frontend test checks heartbeat handling, alert delivery and timer cleanup. These are limited automated checks, not full browser or camera-stream tests.

No load test, penetration test, deployed PostgreSQL integration suite, TLS/deployment audit or end-to-end browser/camera test was carried out in this consistency review. A dated database catalogue inspection appears in section 4.7.1; it is not equivalent to testing PostgreSQL runtime behaviour.

6.5 User Evaluation

None carried out. No officers, administrators or members of the public used the backend directly, and no survey or interview data exist.

6.6 Analysis of Results

- **The intended behaviour is well covered.** The role matrix, ownership rule, lifecycle, cascade delete and plate normalisation all have passing tests, so the core promises of Chapter 4 hold under test.
- **The weaknesses cluster around the token design.** F-1, F-2 and F-7 shared one cause: the server trusted the token's contents (`role`, and the fact that it was issued) and did not re-check the database on each request. That is the stateless trade-off named in §2.2.4. At the evaluated commit it meant a deactivated or demoted police officer kept their old access until the token expired (24 hours by default), which for a police system was the most important finding. The fix removes it for F-1 and F-2 at the price of one database read per request, and leaves F-7 (logout) open, as Table 6.4 shows.
- **F-3 and F-5 are validation gaps.** F-3 concerns status changes by police/admin users. F-5 concerns report creation, which is also available to reporters; it is not limited to police. Both can put the data into states the intended workflow does not expect.
- **F-4 is a consequence of a design choice** (a public endpoint for a camera with no identity). The endpoint cannot alter reports, but anyone who can reach it can flood the alert table and the police dashboard with false alerts for any active plate they know.
- **F-6** matters more if passwords are short, as here.
- **The tests cannot see PostgreSQL-specific problems**, so passing tests are evidence about logic, not about the deployed database.

6.7 Objective-by-Objective Evaluation

Table 6.5: Objective-by-objective evaluation

Objective	Evidence	Verdict
B1 Data model	Four tables with keys, unique email, cascade; tests create the schema and the cascade test passes. <code>create_all()</code> only, no migrations	Achieved , with the migration limitation
B2 Registration, login, hashing, JWT	bcrypt hashes; token with <code>sub</code> , <code>role</code> , <code>exp</code> ; TC-01 to TC-08 pass; deactivation now takes effect at once (Table 6.4)	Achieved ; logout revocation and lockout not provided (F-6, F-7)
B3 Server-side role enforcement	Role dependencies on every route; role matrix TC-09 to TC-17 pass; the role is now read from the database each request (TC-32 to TC-40)	Achieved ; the stale-role weakness (F-1, F-2) is fixed and re-verified
B4 Report lifecycle and ownership	TC-18 to TC-23 pass	Largely achieved ; status validation missing (F-3) and duplicate check incomplete (F-5)
B5 Plate-match endpoint	TC-24 to TC-27 pass	Achieved ; public and unthrottled (F-4)
B6 Audit records	Rows written for selected actions; TC-29 checks one	Largely achieved ; incomplete and non-atomic audit coverage; one audit action directly tested
B7 Verification and security review	40 passing tests; ten findings recorded; probes re-run after the fix	Achieved for this scope; no penetration test

6.8 Individual Results

All results in this chapter concern the backend and security component, except TC-30 and TC-31 (analytics), which are shown for completeness and are not claimed. The camera node, full WebSocket implementation and frontend belong to other work areas. The shared authentication boundary is nevertheless exercised by the backend regression tests; the frontend heartbeat test is reported separately as supporting integration evidence.

6.9 Chapter Summary

All 31 tests at the evaluated commit passed, and all 40 pass after the fix. Six weaknesses were confirmed by probes and one (F-5) by an existing test, the most serious being that tokens outlived deactivation and demotion. Ten findings in total, including three from configuration and code reading, were recorded. F-1 and F-2 were re-checked by the historical probes and retained regression tests; F-9 is verified by configuration/preflight tests; the other seven remain open and are listed as recommendations in Chapter 8.

Chapter 7: Discussion

7.1 Introduction

This chapter interprets the results of Chapter 6, compares them with related practice, states the contributions, and discusses limitations.

7.2 Interpretation of Findings

The access-control design works as specified. Every protected route rejects a missing token, and the role matrix behaves as designed. Reportees cannot read each other's reports, only administrators can delete, and administrators cannot lock themselves out. This answers research question 1 in the affirmative: role-based access can be enforced consistently on the server, using a signed token to identify the account and the current database row to authorise access, without relying on the front end. Public endpoints remain exceptions, including `/system/health`, which exposes operational metrics without a role dependency.

Current account checks fix stale access, but not logout. In the initial evaluation at `a349fc6`, a token retained the role and account access it had when issued. Since `b326b21`, `resolve_token_user` validates the token and re-reads the database user; inactive or deleted accounts are refused, and authorisation uses the current database role. HTTP routes check this on each protected request, while WebSockets check at connection, on received messages and before broadcasts. Logout still only returns a message and invalidates neither the token nor a server-side session. F-1 and F-2 are fixed; F-7 remains open. Database revalidation adds a lookup, while token-specific logout requires a separate revocation mechanism.

Validation is thinner than authorisation. The role checks are consistent, but free-text status and role columns, a plain-string email field and an unvalidated status update let bad values in from people who are already trusted. This is typical of a first version: access control was designed carefully because it is the headline requirement, and data validation was left to the front end.

The public camera endpoint is the sharpest design trade-off. Making check-plate public keeps the camera node simple, and it cannot create or edit reports. But it lets anyone who can reach the server, and who knows or guesses an active plate, generate real-looking alerts on police screens. The risk is bounded by network placement, which the code does not enforce.

7.3 Comparison with Existing Work

No comparison of measured results with other systems was made, because none of the systems reviewed in Chapter 2 publish comparable security test results for a stolen-vehicle registry. On design, this backend implements a small version of what hosted identity providers and mature frameworks provide (hashing, signed tokens, roles) and lacks what they add (revocation, refresh tokens, multi-factor authentication, rate limiting). That is the expected difference between a student prototype and a hardened service. The historical review uses the OWASP Top 10:2021 and API Security Top 10:2023 as organising references [8], [14]. The findings broadly concern access control, authentication, configuration and input validation; this is a qualitative mapping, not an OWASP certification or a complete assessment.

7.4 Contributions of the Project

- **Practical:** a working, documented API with role-scoped access, a report lifecycle, an audit trail and a plate-match endpoint that tolerates formatting differences.
- **Technical:** an access-control layer expressed once as reusable dependencies; a test set-up that runs the production PostgreSQL query on SQLite and checks the cascade.
- **Analytical:** ten recorded findings, supported by the historical probe output, retained regression tests and code inspection. Not every finding came from a probe, and the temporary probe files are not retained.

7.5 Limitations and Implications

Table 7.1: Limitations and implications

Limitation	Implication
Tests run on SQLite	Passing tests do not show PostgreSQL behaviour; the cascade fix and the dated schema catalogue inspection provide limited PostgreSQL evidence, not a PostgreSQL integration suite
Probes were written by the author of the code	They test the weaknesses the author thought of. Other weaknesses may exist
No penetration test or external review	The findings are a lower bound
No load testing	Behaviour under many simultaneous requests, including the <code>async-with-blocking-code</code> concern, is unknown
Severity ratings are the student's own judgement	They are not a formal scoring such as CVSS
Seven findings not fixed	The delivered system still contains F-3 to F-8 and F-10 (§8.5)
No stakeholder input	Requirements may miss needs of real officers

7.6 Chapter Summary

The backend meets its access-control goals, and the tests show it; the stale-token weakness found in the first evaluation was fixed and re-verified. Its remaining weaknesses are concentrated in logout, input validation, login throttling and the public endpoint, and each was demonstrated or read from the code. They are documented here so they can be fixed, not hidden.

Chapter 8: Conclusion and Recommendations

8.1 Introduction

This chapter summarises the project, states the conclusions the evidence supports, reviews the objectives, and gives recommendations and future work.

8.2 Summary of the Project

The group built SVDS so that stolen-vehicle reports filed by the public can be matched automatically against plates seen by a camera. This report covered the backend and security component: a FastAPI service over a four-table PostgreSQL model, with bcrypt password hashing, signed role-carrying tokens, server-side role checks, a report lifecycle, a plate-match endpoint for the camera node, and an audit trail. It was verified by 31 automated tests (40 after a later fix) and by targeted security probes that were re-run after the fix.

8.3 Conclusions

1. Server-side role-based access control with a signed token is achievable with a small amount of code and behaves correctly under test.
2. Passwords are stored as bcrypt hashes and the reportee, police and admin roles are separated as designed.
3. The plate-match endpoint works across case and spacing differences and matches only active reports.
4. The evaluated design had confirmed weaknesses. Tokens outliving deactivation and demotion was fixed and re-verified. Seven of the ten recorded findings remain open: F-3 (status validation), F-4 (public repeated alerts), F-5 (duplicate pending reports), F-6 (login throttling), F-7 (logout), F-8 (default signing-key fallback) and F-10 (administrator seed defaults). Four of these, F-3, F-4, F-6 and F-7, were re-demonstrated in the recorded post-fix probes.
5. The system is a good prototype and is **not** ready for production use with real police data until the recommendations below are addressed.

8.4 Achievement of Objectives

Table 8.1: Achievement of objectives

Objective	Status
B1 Data model	Achieved (no migrations)
B2 Authentication and password storage	Achieved; logout revocation and lockout missing
B3 Server-side role enforcement	Achieved; stale-role weakness fixed and re-verified
B4 Report lifecycle and ownership	Largely achieved; status validation and duplicate check incomplete
B5 Plate-match endpoint	Achieved; public and unthrottled

Objective	Status
B6 Audit records	Largely achieved; failed logins not recorded
B7 Verification and security review	Achieved for this scope

Research question 1: yes. Research question 2: ten weaknesses were found and recorded; three (F-1, F-2, F-9) have been fixed and re-verified, seven are open.

8.5 Recommendations

In priority order:

1. **Done for deactivation and demotion (F-1, F-2):** protected requests now re-check the user. **Still to do for logout (F-7):** implement token revocation (for example a deny-list or session version). Short token lifetimes and refresh-token rotation reduce exposure but do not by themselves invalidate an issued access token immediately.
2. **Validate report status** with an enumeration in the schema and a database check constraint, and route status changes through the lifecycle endpoints only (F-3). Add a database constraint for role as defence in depth; registration and admin role changes already restrict values in route code.
3. **Protect** check-plate with an API key or a private network, and add rate limiting and de-duplication of repeated alerts (F-4).
4. **Throttle failed logins**, log them in the audit trail, and adopt an appropriate minimum length and a common/compromised-password block-list; avoid mandatory character-type composition rules (F-6) [15].
5. **Fail at start-up if SECRET_KEY is unset** instead of using a default (F-8), (allowed origins are now read from configuration, F-9).
6. Remove the default password from `create_admin.py`, require `ADMIN_PASSWORD`, and stop printing it (F-10).
7. Extend the duplicate check to `under_review` reports (F-5).
8. Adopt Alembic migrations and run the test suite against PostgreSQL.
9. Validate email addresses, and apply the registration formats to profile edits.
10. Write the audit row in the same transaction as the action it records.

8.6 Future Work

- Fix the open findings above and turn each remaining probe into a permanent regression test, as was done for F-1 and F-2.
- Add multi-factor authentication for police and admin accounts.
- Add a structured penetration test and a load test.
- Add pagination totals and stronger search to the report lists.
- Move blocking database calls out of `async` routes, or use an `async` database driver.
- Define data retention and access-request rules that meet Zambian data protection requirements.

References

The report uses numbered references. Online sources were checked on 25 September 2026. Versioned documentation is cited where available; the installed versions used for verification are recorded in Chapter 6. OWASP 2021 and 2023 are the identified review baselines, not a claim that they are the latest editions.

1. FastAPI, *FastAPI Documentation: dependencies, CORS and testing*. [Official documentation](#). Accessed 25 September 2026.
2. SQLAlchemy, *SQLAlchemy 2.0 Documentation*, "Describing Databases with MetaData." [Schema documentation](#). Accessed 25 September 2026.
3. PostgreSQL Global Development Group, *PostgreSQL 18 Documentation*. [Version 18 documentation](#). Accessed 25 September 2026.
4. Pydantic, *Pydantic 2.7 Documentation*. [Version 2.7 documentation](#). Accessed 25 September 2026.
5. M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015. [RFC 7519](#).
6. N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," in *1999 USENIX Annual Technical Conference*, FREENIX Track, 1999, pp. 81-91. [USENIX paper record](#).
7. Passlib, *Passlib 1.7.4 Documentation*. [Version 1.7.4 documentation](#). Accessed 25 September 2026.
8. OWASP Foundation, *OWASP Top 10:2021*. [2021 edition](#). Accessed 25 September 2026.
9. D. Ferraiolo and D. R. Kuhn, "Role-Based Access Controls," in *Proceedings of the 15th National Computer Security Conference*, Baltimore, MD, 1992, pp. 554-563. [NIST publication record](#).
10. R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, Ph.D. dissertation, University of California, Irvine, 2000. [University-hosted dissertation](#).
11. pytest, *pytest Documentation*. [Official documentation](#). Accessed 25 September 2026.
12. Django REST framework, "Authentication," *Django REST Framework Documentation*. [Authentication guide](#). Accessed 25 September 2026.
13. Keycloak, *Server Administration Guide*. [Official administration guide](#). Accessed 25 September 2026.
14. OWASP Foundation, *OWASP API Security Top 10 - 2023*. [2023 edition](#). Accessed 25 September 2026.
15. National Institute of Standards and Technology, *Digital Identity Guidelines: Authentication and Authenticator Management*, NIST SP 800-63B-4, 2025, section 3.1.1. [Password guidance](#). Accessed 25 September 2026.
16. Republic of Zambia, *The Data Protection Act, 2021*, Act No. 3 of 2021. [National Assembly record](#). Accessed 25 September 2026.

Appendix A: Individual Contribution Statement (Group Projects)

Purpose of this Statement

This statement identifies the work for which the student was primarily responsible. It is consistent with the main report. Work by other group members is acknowledged and is not claimed.

A.1 Project and Group Information

Table A.1: Project and group information

Item	Details
Project Title	SVDS: A Camera-Based Stolen Vehicle Detection System for the Zambia Police Service (built on the VRRS registry)
Student Name	Moomba Nicholas Katapazi
Computer Number	2021473333
Supervisor	[SUPERVISOR NAME]
Group Size	4
Group Members	Backend and security: Moomba Nicholas Katapazi, 2021473333. AI/vision, real-time/integration and frontend members: details pending author input.

A.2 Overall Group Project

The group built **SVDS/VRRS**, a system with which the public files stolen-vehicle reports and police officers are alerted automatically when a camera sees a reported vehicle. It has three independently running processes: a FastAPI and PostgreSQL backend, a React frontend with three role-based portals (Reportee, Police, Admin), and a camera node that reads number plates from a phone's video stream using a custom-trained YOLOv8 detector and OCR. The common problem is that stolen-vehicle recovery depends on officers chancing to recognise a plate. The system's core flow is: camera node reads a plate, the backend matches it against active reports, and connected police dashboards receive an alert over WebSocket.

A.3 Individual Role and Responsibilities

The group's work was divided into four roles. The student's role was **Role 2: Backend and security**, which covers work areas 3 (backend API and database) and 4 (security and access control).

Table A.2: Individual role and responsibilities

No.	Area of Responsibility	Description
3	Backend API and database	FastAPI service and PostgreSQL schema; report CRUD and lifecycle; the /alerts/check-plate matching endpoint

No.	Area of Responsibility	Description
4	Security and access control	JWT login, bcrypt password hashing, Reportee/Police/Admin roles and middleware, user management, audit records

Other roles, for reference: **Role 1** the plate-detection model and camera node; **Role 3** WebSocket alerts, camera status and live-feed proxy, system health, and the phone-to-server wiring; **Role 4** the React portals, analytics, integration tests, run scripts and documentation.

A.4 Detailed Individual Contribution

- **Data design:** designed the four-table schema (users, reports, alerts, audit log), keys, unique email and the cascade from reports to alerts.
- **Software development:** wrote the FastAPI application structure, database session handling, ORM models and Pydantic schemas with telephone and national ID formats.
- **Security:** implemented bcrypt password storage, JWT issue and checking, and the `require_role` dependencies; implemented ownership rules on reports and the admin safeguards on user management.
- **Business logic:** implemented the report lifecycle (`under_review` to `missing` to `found`), the duplicate-active-report rule, and the plate-match endpoint with case and spacing normalisation.
- **Auditing:** added audit records to state-changing actions.
- **Defect fixing:** fixed the report-delete cascade defect.
- **Testing and security review:** ran the 31-test suite, wrote and ran the security probes, and recorded ten findings with recommendations.
- **Documentation:** this report, including the API and role tables.

A.5 Individual Deliverables

Table A.3: Individual deliverables

No.	Deliverable	Student's Contribution
1	Database schema and ORM models (<code>models.py</code> , <code>database.py</code>)	Designed and implemented
2	Authentication and access control (<code>auth.py</code> , <code>middleware.py</code> , <code>routers/auth.py</code> , <code>routers/users.py</code>)	Designed and implemented
3	Report lifecycle and plate-match endpoint (<code>routers/reports.py</code> , <code>check-plate</code> in <code>routers/alerts.py</code>)	Designed and implemented
4	Request and response schemas (<code>schemas.py</code>)	Designed and implemented
5	Security review: probe results and findings F-1 to F-10	Carried out and documented

A.6 Contribution to the Main System or Research Output

The student produced the server that stores and protects the system's data and answers the camera's question "is this plate stolen?". The camera node itself was built by the AI/vision member; the WebSocket delivery, camera status and live-feed proxy by the real-time and integration member; the web pages and analytics by the frontend member. The check-plate request and response format was agreed with the AI/vision member. The WebSocket manager lives in the same file as check-plate (routers/alerts.py) and was written by the real-time member.

A.7 Individual Implementation and Technical Work

Modules implemented by the student: main.py, database.py, models.py, schemas.py, auth.py, middleware.py, routers/auth.py, routers/users.py, routers/reports.py, and the check-plate and alert-listing routes in routers/alerts.py. Databases and integrations: PostgreSQL schema, the camera-node contract, and the hook that calls the real-time broadcast.

A.8 Individual Testing and Evaluation Contribution

The student ran the 31-test backend suite, wrote and ran the security probe tests, interpreted the results, and wrote Chapters 6 and 7. **Authorship of each file in vrrs-backend/tests/ should be confirmed by the student:** the group's role split placed "tests" with the frontend member, so this report does not claim the suite as the student's own work. The camera-node tests and the manual browser checks belong to other members.

A.9 Collaboration with Other Group Members

- The **AI/vision member's** camera node calls POST /alerts/check-plate; its request and response format was agreed jointly.
- The **real-time and integration member's** WebSocket manager is called by check-plate and reuses the shared resolve_token_user check; the WebSocket admits only police and admin users.
- The **frontend member's** pages call every route in Chapter 4 and depend on the role and redirect returned at login.

A.10 Contribution Summary

1. **Work primarily completed by the student:** the data model, authentication and access control, the report lifecycle, the plate-match endpoint, audit records and the security review.
2. **Work completed jointly with other members:** the camera-node contract; the alert broadcast hook; start-up through run-all.ps1.
3. **Work primarily completed by other members and used here:** the detection model and camera node; the WebSocket delivery, camera status and health endpoints; the web portals and analytics.

A.11 Estimated Contribution

The percentages below are the student's own estimate and must be completed by the student. They were not derived from any record.

Table A.4: Estimated contribution

Project Activity	Individual Contribution	Collaborative Contribution
Problem Analysis	[%]	[%]
Literature Review	[%]	[%]
Requirements Analysis	[%]	[%]
System/Research Design	[%]	[%]
Implementation / Development	[%]	[%]
Testing and Evaluation	[%]	[%]
Documentation	[%]	[%]
Other	[%]	[%]

A.12 Statement of Authorship and Contribution

I confirm that the contribution described in this statement accurately represents the work that I personally undertook as part of the group project. I have identified collaborative work and have not knowingly claimed the work of another group member as my own.

Student Name: Moomba Nicholas Katapazi

Computer Number: 2021473333

Signature: _____

Date: _____

A.13 Supervisor Verification

I confirm that, to the best of my knowledge, the contribution described above is consistent with the student's participation in the group project.

Supervisor Name: _____

Signature: _____

Date: _____

Appendix B: User Manual / Installation Guide

This guide covers the backend. The camera node and front end are documented in the other members' reports and the project README.md.

B.1 Requirements

- Python 3.12.10 (the tested runtime), and PostgreSQL with an empty database (for example vrrs_db).
- The packages in vrrs-backend/requirements.txt.

B.2 Install and configure

```
cd vrrs-backend
pip install -r requirements.txt
cp .env.example .env
```

Edit .env:

Table B.1: Install and configure

Key	Meaning	Note
DATABASE_URL	PostgreSQL connection string	e.g. postgresql://postgres:<password>@localhost:5432/vrrs_db
SECRET_KEY	Token signing key	Must be a long random value. The code falls back to changeme if it is missing (finding F-8)
ALGORITHM	JWT algorithm	HS256
TOKEN_EXPIRY_HOURS	Token lifetime	24 by default
CORS_ORIGINS	Comma-separated browser origin allow-list	Read by get_allowed_origins; an absent value enables four localhost defaults; an empty value permits no origin. F-9 is fixed

B.3 Run

```
uvicorn app.main:app --reload
```

The API runs at http://localhost:8000. Interactive documentation is at /docs. Tables are created automatically at start-up if they do not exist.

Public registration creates reportees only, so the first administrator is created with the helper script create_admin.py:

```
$env:ADMIN_EMAIL = "admin@example.local"
$env:ADMIN_PASSWORD = Read-Host "Administrator password"
python create_admin.py
```

Always set ADMIN_PASSWORD yourself. If it is unset the script uses a default password that is written in its source (finding F-10). The script also defaults to a local SQLite file when DATABASE_URL is not set,

so set that too. Later role changes are made by an administrator, and a promoted account must have a badge number.

B.4 Roles at a glance

Table B.2: Roles at a glance

Role	Can do
Reportee	Register, log in, file reports, see own reports, edit own profile
Police	Everything a reportee can, plus see all reports and users, activate and mark reports found, edit reports, view and flag alerts, view analytics
Admin	Everything police can, plus create police accounts, change roles, deactivate or delete users, delete reports, read the audit log

B.5 Troubleshooting

Table B.3: Troubleshooting

Symptom	Likely cause	Action
401 Invalid or expired token	Missing, expired or wrongly signed token	Log in again; check SECRET_KEY has not changed since the token was issued
403 Access denied	Role too low for the route	Use an account with the required role
409 when filing a report	A missing report for that plate exists	Check the existing report
Start-up prints "error creating DB schema"	Cannot reach PostgreSQL	Check DATABASE_URL and that PostgreSQL is running
Frontend blocked by CORS	Its origin is not in the configured allow-list	Set CORS_ORIGINS and restart the backend
Deleting a report fails on an older database	The cascade constraint was created before the fix	Alter the alerts.report_id foreign key to ON DELETE CASCADE by hand

B.6 Run the tests

```
cd vrrs-backend
python -m pytest tests/ -v
```

The tests use an in-memory SQLite database and do not touch PostgreSQL.

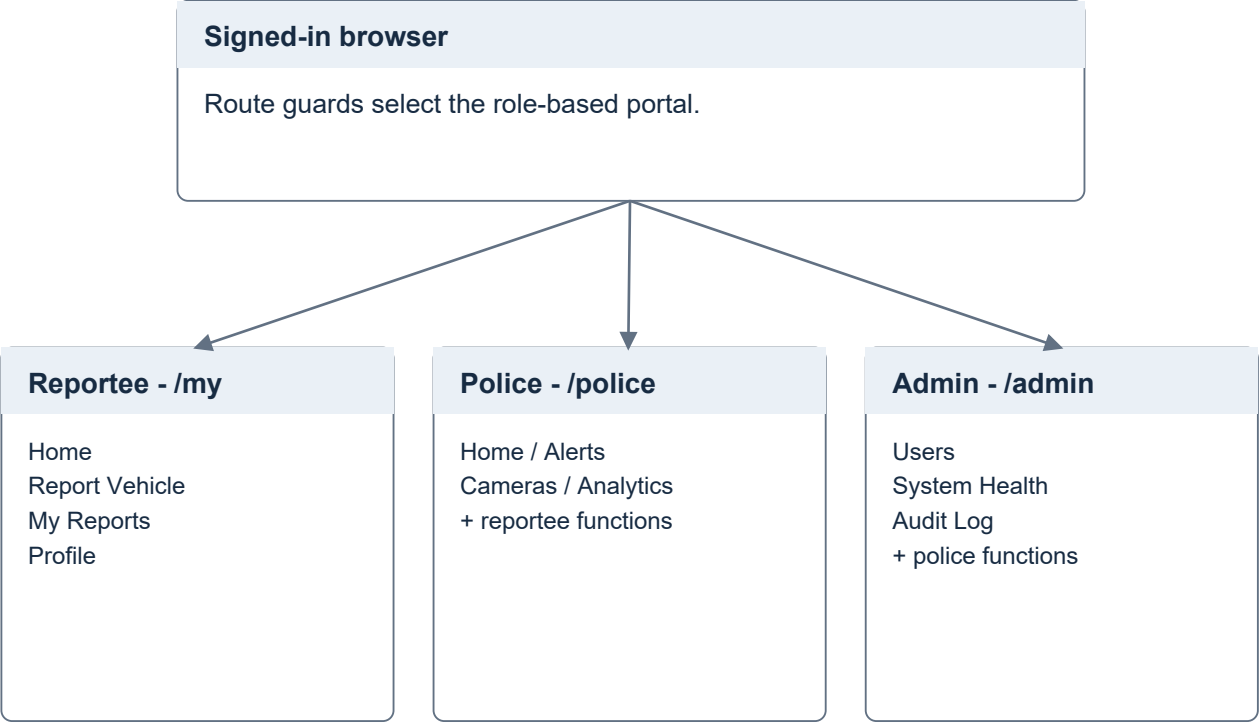
Appendix C: Additional System Designs / Diagrams

C.1 Authentication sequence



Figure C.1: authentication sequence

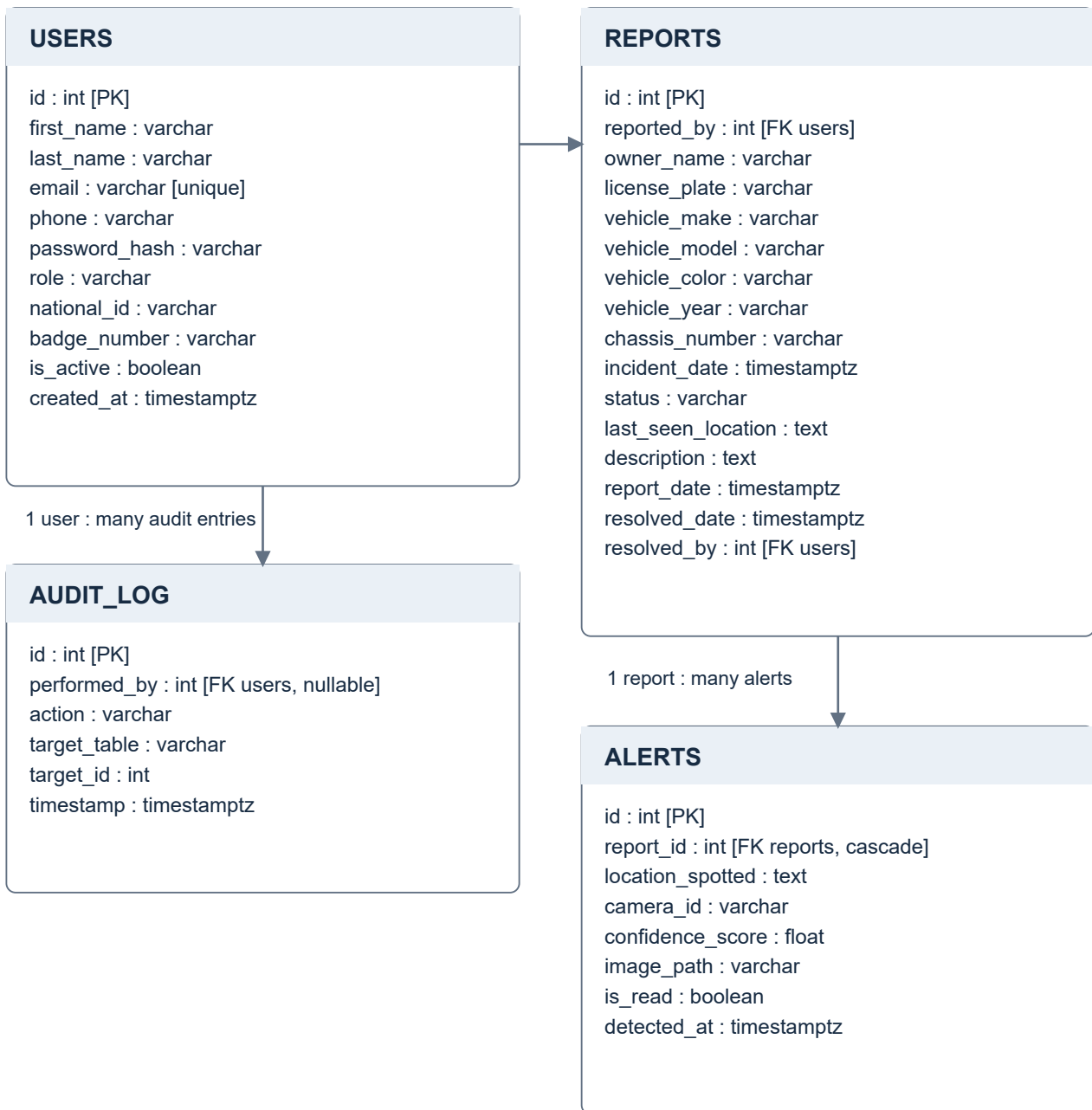
C.2 Role-based route access



The API independently checks the current database user and permitted role.

Figure C.2: role-based route access

C.3 Database schema



Users file and resolve reports (1 : many). PK = primary key; FK = foreign key.

timestampz = timestamp with time zone; timestamp = without time zone.

Figure C.3: entity-relationship diagram

C.4 Detection-to-alert sequence

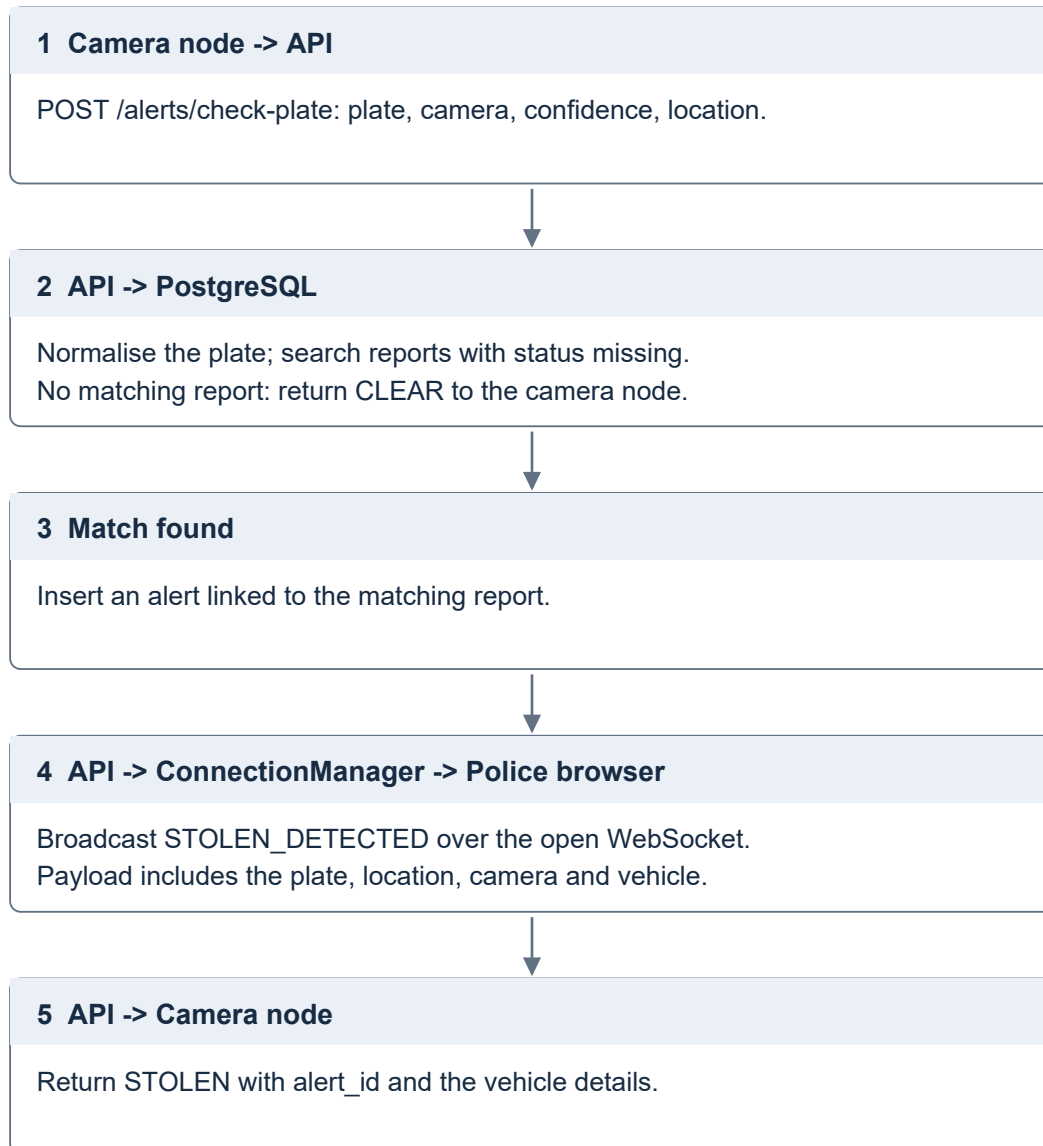


Figure C.4: detection-to-alert sequence

Appendix D: Additional Test Cases and Results

D.1 Raw test output

The full captured output of the 40 passing tests (commit b326b21) is in thesis/assets/test-results/backend-pytest-output.txt. The 31 tests at the evaluated commit a349fc6 also passed when the report was first prepared.

D.2 Probe results (raw)

The probe tests were temporary and were deleted after running; their printed output is reproduced below. These are the results at the evaluated commit a349fc6; the results after the fix are in D.2a.

```
PROBE deactivated-user token /users/me          -> 200
PROBE demoted-admin old token /system/audit     -> 200
PROBE PATCH status=banana                      -> 200 banana
PROBE check-plate no-auth x3                   -> ['STOLEN','STOLEN','STOLEN'] alerts rows: 3
PROBE 20 wrong logins status set               -> {401}
PROBE register password 'abcdef'              -> 422 (invalid probe: payload incomplete)
PROBE default-key forged token accepted        -> False
PROBE SECRET_KEY equals 'changeme'            -> False | length 47
PROBE token after /auth/logout                 -> 200
```

D.2a Probe results after the fix (raw)

The same scenarios re-run against commit b326b21 (temporary test file, deleted afterwards). Only the fields that matter are shown.

```
F1 deactivated-user token /users/me          before deactivation 200 | after 401
F2 demoted-admin old token /system/audit     before demotion 200    | after 403
F3 PATCH status=banana                      -> 200
F3 PATCH status=missing                     -> 200
F4 check-plate no-auth x3                   -> STOLEN, STOLEN, STOLEN (alert ids 1, 2, 3)
F6 20 wrong logins status set               -> {401}
F7 token after /auth/logout                 -> 200 (the logout call itself: 200)
F8 SECRET_KEY equals 'changeme'            -> False (test configuration key)
```

The F-1 and F-2 lines are the only ones that changed.

D.3 One probe as an example

```
def test_probe_deactivated_token_still_works(client, make_user, auth_headers, db_session):
    u = make_user(role="reportee"); h = auth_headers(u)
    u.is_active = False; db_session.commit()
    print(client.get("/users/me", headers=h).status_code)          # observed: 200
    assert client.post("/auth/login",
        json={"email": u.email, "password": "Password123"}).status_code == 403
```

Appendix E: Data Collection Instruments

Not applicable. No questionnaires, interviews or surveys were used.

Appendix F: Additional Code / Configuration

F.1 Role dependencies (app/middleware.py)

```
# As evaluated at commit a349fc6 (the source of F-1 and F-2). Since b326b21 the
# body looks the user up in the database instead (see section 5.4.2).
def get_current_user(token: str = Depends(oauth2_scheme)):
    payload = decode_token(token)
    if not payload:
        raise HTTPException(status_code=401, detail="Invalid or expired token.",
                             headers={"WWW-Authenticate": "Bearer"})
    return {"id": int(payload["sub"]), "role": payload["role"]}

def require_role(*roles):
    def checker(user: dict = Depends(get_current_user)):
        if user["role"] not in roles:
            raise HTTPException(status_code=403,
                                detail=f"Access denied. Required roles: {list(roles)}")
        return user
    return checker
```

F.2 Where finding F-1 and F-2 come from

At the evaluated commit, `get_current_user` returned the role from the token and never read the database, so a changed role or a deactivated account was not seen until the token expired. The fix, applied in b326b21, looks up User by id here and rejects missing or inactive users (§5.4.2).

F.3 Environment template (.env.example)

```
DATABASE_URL=postgresql://postgres:yourpassword@localhost:5432/vrrs_db
SECRET_KEY=your-super-secret-key-change-this-in-production
ALGORITHM=HS256
TOKEN_EXPIRY_HOURS=24
CORS_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
```

Appendix G: Other Supporting Material

G.1 Group role split

Table G.1: Group role split

Role	Work areas	Focus
1. AI/vision	Detection model; camera node	YOLOv8n detector, OCR pipeline
2. Backend and security	Backend API and database; security and access control	This report
3. Real-time and integration	Real-time alerts and monitoring; phone-to-server wiring	
4. Frontend	Frontend portals; analytics, testing and documentation	React pages, analytics, run scripts

G.2 Findings summary

Table G.2: Findings summary

ID	Finding	Severity at initial evaluation	Current status
F-1	Token valid after deactivation	High	Fixed
F-2	Token keeps old role after demotion	High	Fixed
F-3	Report status accepts any value	Medium	Open
F-4	Public check-plate creates repeat alerts	Medium	Open
F-5	Duplicate check ignores under_review	Low	Open
F-6	No limit on failed logins	Medium	Open
F-7	Logout does not invalidate token	Medium	Open
F-8	Default signing key fallback (latent)	Latent	Open
F-9	CORS_ORIGINS setting unused	Low	Fixed
F-10	create_admin.py default password in source	Medium	Open